The ACM Conference Series on Recommender Systems

Conversational Recommender System Using Deep Reinforcement Learning

Omprakash Sonie (DeepThinking.AI, India)

Agenda

1. Deep Reinforcement Learning Foundation (20min)
• Foundations for RL, Deep Q-Network, Dyna, REINFORCE and Actor Critic [1][2]

2. DRL approaches for CRS - I (20min)
• Conversational Recommender System [3]
• Towards Deep Conversational Recommendations [4]
• Deep Interaction between Conversational and Recommender Systems [5]
• Learning and Adaptivity in Interactive Recommender Systems [6]

3. DRL approaches for CRS - II (20min)
• Towards Topic-Guided Conversational Recommender System [7]
• Interactive Path Reasoning on Graph for Conversational Recommendation [8]
• Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning [9]
• Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning [10]

4. DRL approaches for CRS - III (20min)
• Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender [11]
• Large-scale Interactive Recommendation with Tree-structured
Policy Gradient [12]
• A Reinforcement Learning Framework for Interactive Recommendation [13]
• Large-scale Interactive Conversational Recommendation System
using Actor-Critic Framework [14]

5. CONCLUSION (5min)

2 DRL approaches for CRS - I (20min)

Conversational Recommender System [3]

- Towards Deep Conversational Recommendations [4]
- Deep Interaction between Conversational and Recommender Systems [5]
- Learning and Adaptivity in Interactive Recommender Systems [6]

2.2 Towards Deep Conversational Recommendations

There has been growing interest in using neural networks and deep learning techniques to create dialogue systems. Conversational recommendations is an interesting setting for the scientific exploration of dialogue with natural language as the associated discourse involves goal-driven dialogue that often transforms naturally into more free-form chat. This paper provides two contributions. First, until now there has been no publicly available large-scale dataset consisting of real-world dialogues centered around recommendations. To address this issue and to facilitate our exploration here, we have collected REDIAL, a dataset consisting of over 10,000 conversations centered around the theme of providing movie recommendations. We make this data available to the community for further research. Second, we use this dataset to explore multiple facets of conversational recommendations. In particular we explore new neural architectures, mechanisms, and methods suitable for composing conversational recommendation systems. Our dataset allows us to systematically probe model sub-components addressing different parts of the overall problem domain ranging from: sentiment analysis and cold-start recommendation generation to detailed aspects of how natural language is used in this setting in the real world. We combine such sub-components into a full-blown dialogue system and examine its behavior.

[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018

2.2 Towards Deep Conversational Recommendations

Conversational recommendation:

- involves goal-driven dialogue that often transforms naturally into more free-form chat

Contribution:

1 No publicly available large-scale dataset consisting of real-world dialogues centered around recommendations.

- Collected REDIAL, a dataset consisting of over 10,000 conversations centered around the theme of providing movie recommendations.
- Make this data available to the community for further research.

2 Use this dataset to explore multiple facets of conversational recommendations.

- Explore new neural architectures, mechanisms, and methods suitable for composing conversational recommendation systems.

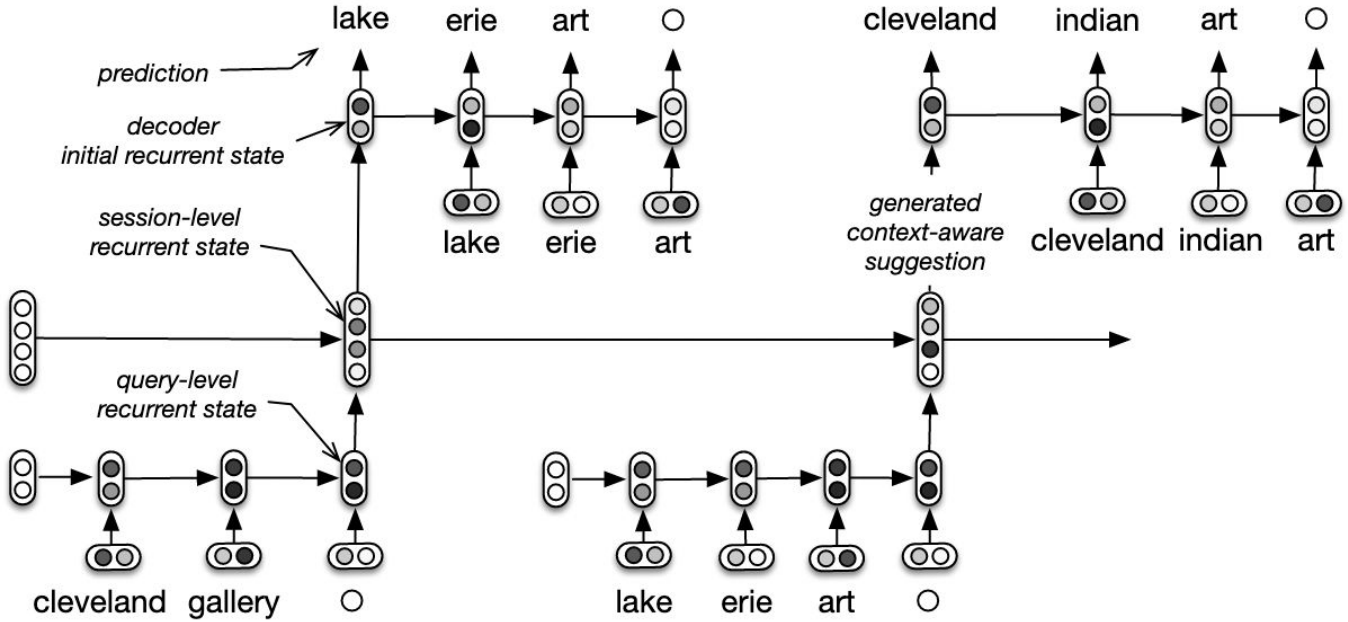
Dataset allows to systematically probe model sub-components addressing different parts of the overall problem domain ranging from:

- Sentiment analysis and Cold-start recommendation generation to
    - detailed aspects of how natural language is used in this setting in the real world

Combine such sub-components into a full-blown dialogue system and examine its behavior

[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018

2.2 Towards Deep Conversational Recommendations



REF: The hierarchical recurrent encoder-decoder (HRED) for query suggestion. Each arrow is a non-linear transformation. The user types **cleveland gallery → lake erie art**. During training, the model encodes **cleveland gallery,** updates the session-level recurrent state and maximize the probability of seeing the following query **lake erie art.**
The process is repeated for all queries in the session. During testing, a contextual suggestion is generated by encoding the previous queries, by updating the session-level recurrent states accordingly and by sampling a new query from the last obtained session-level recurrent state. In the example, the generated contextual suggestion is *cleveland indian art.*

[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018

2.2 Towards Deep Conversational Recommendations

# conversations	10021
# utterances	163820
# users	963
# movie mentions	51892
seeker mentioned	16338
recommender suggested	35554
not seen	16571
seen	31818
did not say	3503
disliked (4.9%)	2567
liked (81%)	42152
did not say (14%)	7173

Table 1a. REDIAL data statistics. For the movie dialogue forms, the numbers shown represent the seeker's answers.

HUMAN:	hello, can i help you find a movie ?
HRED:	hi what kind of movies do you like
OURS:	hi, how can i help you ?
SEEKER:	hello ! i would like to find a suspenseful, but clean family friendly movie .
...	...
SEEKER:	2001 : a space odyssey might be a great option. anything else that you would suggest ?
HUMAN:	you can try planet of the apes the older one is quite suspenseful and family friendly .
HRED:	i haven't seen that one yet but i heard it was good. i haven't seen that one. have you seen the last house on the left ?
OURS:	star wars : the force awakens is also a good one return of the jedi all good movies
SEEKER:	those sound good ! i 'm going to look into those movies.
HUMAN:	i hope you enjoy, have a nice one
HRED:	have you seen foxcatcher ? it 's about a man who has a rich guy.
OURS:	i hope i was able to help you find a good movie to watch
SEEKER:	thank you for your help ! have a great night ! good bye

Table 1b.Conversation excerpts (HUMAN followed by response by SEEKER) and model outputs (OUR proposed approach compared to HRED a generic dialogue model [*Sordoni A. et. al., A hierarchical recurrent encoder-decoder for generative context-aware query suggestion*]).

[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018

2.2 Towards Deep Conversational Recommendations

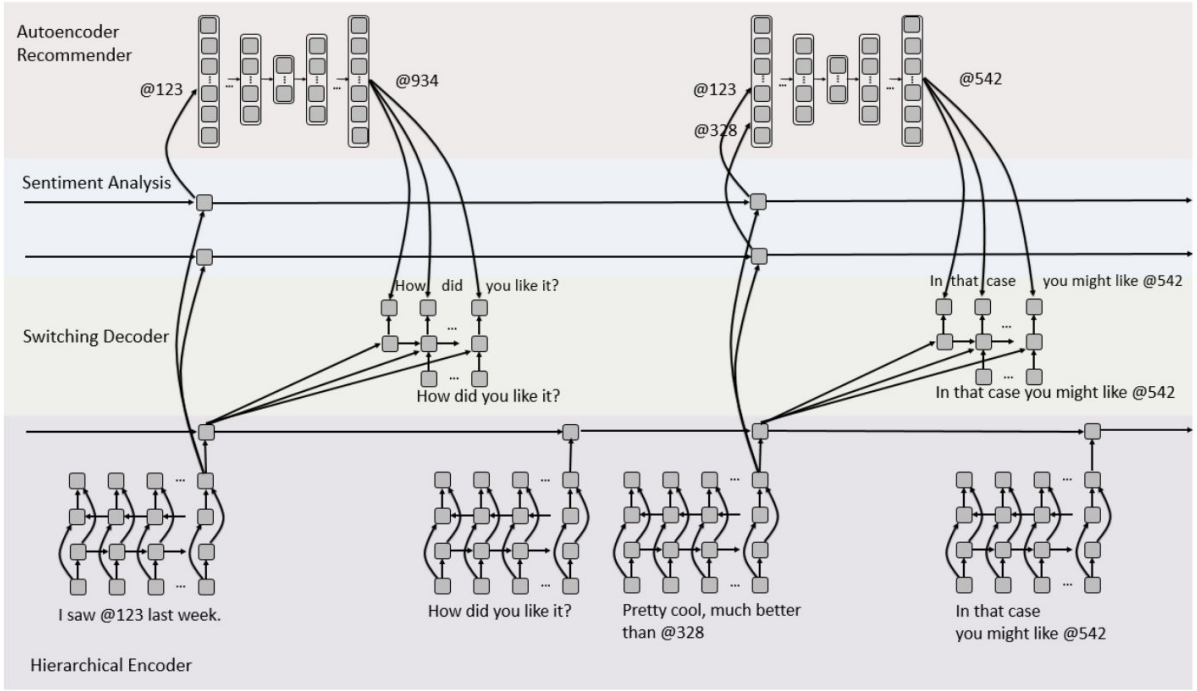


Figure 1: Proposed model for conversational recommendations.

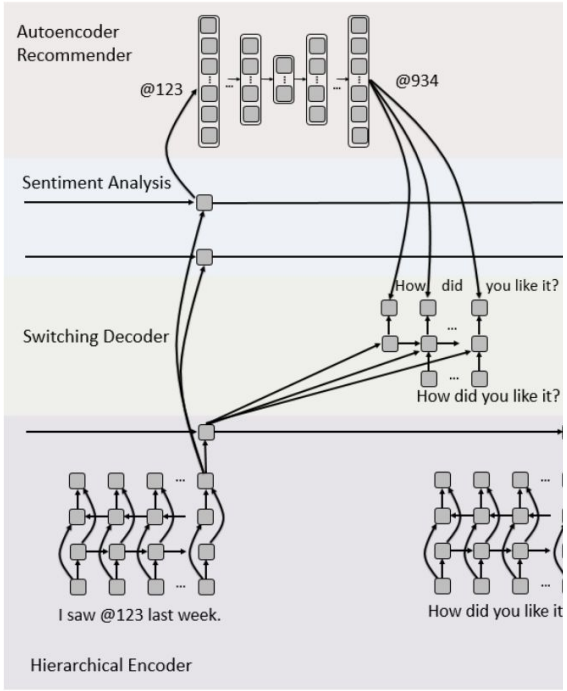
1. **A hierarchical recurrent encoder** following the HRED architecture

2. **A switching decoder:** model the dialogue acts generated by the recommender

3. After each dialogue act model detects if a movie entity has been discussed (with the @identifier convention) and
   - instantiate an RNN focused on classifying the seeker's sentiment or opinion regarding that entity.
   - The sentiment analysis RNNs are used to indicate the user opinions forming the input to AE based recommender module

4. **Autoencoder-based recommendation module** The AE's output is used by the decoder through a switching mechanism.
   - Some of these components can be pre-trained on external data, thus compensating for the small data size.
   - The switching mechanism allows one to include the recommendation engine, which we trained using the significantly larger MovieLens data.

2.2 Towards Deep Conversational Recommendations



1. Hierarchical recurrent encoder: each utterance U_m as a sequence of N_m words $U_m = (w_{m,1}, ..., w_{m,N_m})$ where the tokens $w_{m,n}$ are either words from a vocabulary V or movie names from a set of movies V'. Use a scalar $s_m \in \{-1, 1\}$ appended to each utterance to indicate the role (recommender or seeker) such that a dialogue of M utterances can be represented as $D = ((U_1, s_1), ..., (U_M, s_M))$.

Use a GRU to encode utterances and dialogues. Given an input sequence $(i_1, ..., i_T)$, the network →computes reset gates r_t, input gates z_t, new gates n_t and forward hidden state h_t as follows

$$\mathbf{r}_t = \sigma\left(\mathbf{W}_{ir}\mathbf{i}_t + \mathbf{W}_{hr}\overrightarrow{\mathbf{h}}_{t-1} + \mathbf{b}_r\right), \quad \mathbf{z}_t = \sigma\left(\mathbf{W}_{iz}\mathbf{i}_t + \mathbf{W}_{hz}\overrightarrow{\mathbf{h}}_{t-1} + \mathbf{b}_z\right)$$

$$\mathbf{n}_t = \tanh\left(\mathbf{W}_{in}\mathbf{i}_t + \mathbf{b}_{in} + \mathbf{r}_t \circ (\mathbf{W}_{hn}\overrightarrow{\mathbf{h}}_{t-1} + \mathbf{b}_{hn})\right), \quad \overrightarrow{\mathbf{h}}_t = (1 - \mathbf{z}_t) \circ \mathbf{n}_t + \mathbf{z}_t \circ \overrightarrow{\mathbf{h}}_{t-1}$$

W** and b* are learned parameters

2. Dynamically Instantiated RNNs for Movie Sentiment Analysis

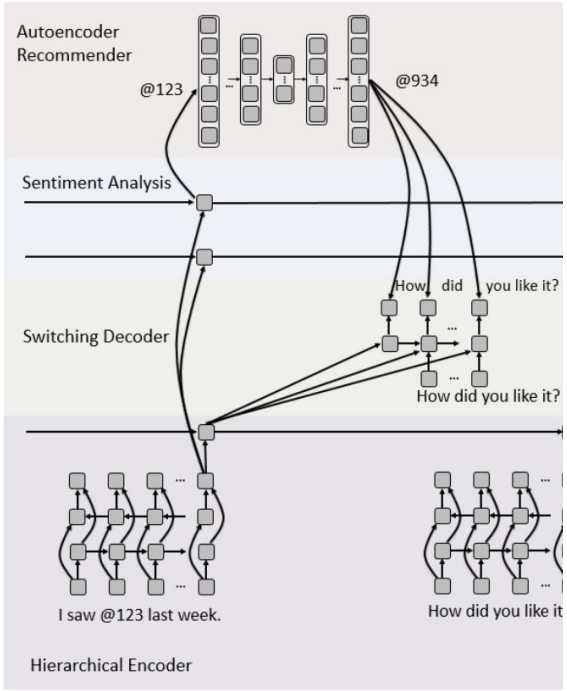
the model predicts different answers for the seeker and for the recommender. For each participant it learns to predict three labels:

- the “suggested” label (binary), the “seen” label (categorical with three classes), the “liked” label (categorical with three classes)
- $D = \{(x_i, y_i), i = 1..N\}$ the training set, where $x_i = (D_i, m_i)$ is the pair of a dialogue D_i and a movie name m_i that is mentioned in D_i and

$$y_i = (\underbrace{y_i^{\text{sugg}}, y_i^{\text{seen}}, y_i^{\text{liked}}}_{\text{seeker's answers}}, \underbrace{y_i'^{\text{sugg}}, y_i'^{\text{seen}}, y_i'^{\text{liked}}}_{\text{recommender's answers}}), \quad (1)$$

yi are the labels in the movie dialogue form corresponding to movie m_i in dialogue D_i

2.2 Towards Deep Conversational Recommendations



3. The Autoencoder Recommender

At the start of each conversation, the recommender has no prior information on the movie seeker (cold start).

During the conversation, the recommender gathers information about the movie seeker and (implicitly) builds a profile of the seeker's movie preferences.

User-based autoencoder for collaborative filtering (U-Autorec), a model capable of predicting ratings for users not seen in the training set.

Use a similar model and pre-train it with MovieLens data.

4 Decoder with a Movie Recommendation Switching Mechanism

The sentiment analysis RNNs presented above predict for each movie mentioned so far whether the seeker liked it or not using the previous utterances.

- Decodes the context to predict the next utterance step by step

Switching mechanism allows to include an explicit recommendation system in the dialogue agent.

[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018

2.4 Learning and Adaptability in Interactive Recommender System

Recommender systems are intelligent E-commerce applications that assist users in a decision-making process by offering personalized product recommendations during an interaction session. Quite recently, conversational approaches have been introduced in order to support more interactive recommendation sessions. Notwithstanding the increased interactivity offered by these approaches, the system employs an interaction strategy that is specified apriori (at design time) and followed quite rigidly during the interaction. In this paper, we present a new type of recommender system which is capable of *learning autonomously* an adaptive interaction strategy for assisting the users in acquiring their interaction goals. We view the recommendation process as a sequential decision problem and we model it as a Markov Decision Process (MDP). We learn a model of the user behavior, and use it to acquire the adaptive strategy using Reinforcement Learning (RL) techniques. In this context, the system learns the optimal strategy by observing the consequences of its actions on the users and also on the final outcome of the recommendation session. We apply our approach within an existing travel recommender system which uses a rigid, non-adaptive support strategy for advising a user in refining a query to a travel product catalogue. The initial results demonstrate the value of our approach and show that our system is able to *improve* the non-adaptive strategy in order to learn an optimal (adaptive) recommendation strategy.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Intelligent E-commerce applications

- Assist users in a decision-making process by offering personalized product recommendations during an interaction session.
- Support more interactive recommendation sessions system by employing an interaction strategy that is specified apriori (at design time) and followed quite rigidly during the interaction.

The recommender system which is

- Capable of *learning autonomously* an adaptive interaction strategy for assisting the users in acquiring their interaction goals.

Recommendation process

- as a sequential decision problem and
- model it as a Markov Decision Process (MDP).
- learn a model of the user behavior, and use it to acquire the adaptive strategy using Reinforcement Learning (RL) techniques.
- the system learns the optimal strategy by observing the consequences of its actions on the users and also on the final outcome of the recommendation session.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

S*trategy:* the abstract plan of action which the system employs during the interaction session.

System may employ the following two strategies:
1. Strategy 1: start querying the user about her preferences, and acquire enough information from her, in order to select a candidate set of hotels, or

2. Strategy 2: initially propose some candidates and acquire user preferences as critiques (a critique is a special type of user feedback) in order to personalize the future recommendations.

Different policies are learned when the agent estimates in a different way

- the cost of each interaction, i.e., the dissatisfaction of the user to not have reached the goal, which, in this case, is the selection of the best product
- the RL based discount rate parameter γ, that models the probability of the user being willing (or unwilling) to continue the interaction.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

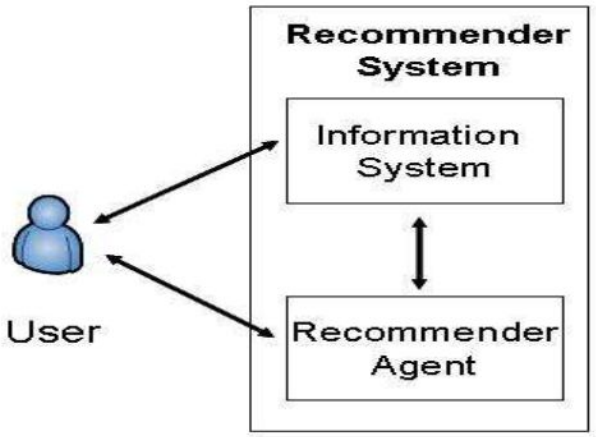


Figure 1: Adaptive Recommender Model

Information System (IS): Non adaptive - what user does

Recommendation Agent (RA) is the adaptive entity whose role is to
- observe the user, the IS and
- the ongoing interaction in order to support the user in his decision task,
- by helping him to obtain the right information at the right time

In a travel agency scenario,
- a traveller browses a catalogue (IS) to get suggestions/recommendations, and
- the travel agent (RA) helps the user to find suitable products by asking questions or pointing to some catalogue pages or products

Interactive personalisation process:
- understand customers by collecting information about them
- deliver personalized offerings based on this information
- measure the personalization impact by determining the user satisfaction with these offerings.

Goal is to learn the best strategy to deliver personalized offerings to the users, i.e., decide when to push some recommendation, acquire some user characteristics, pop-up some information, show most popular products etc.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

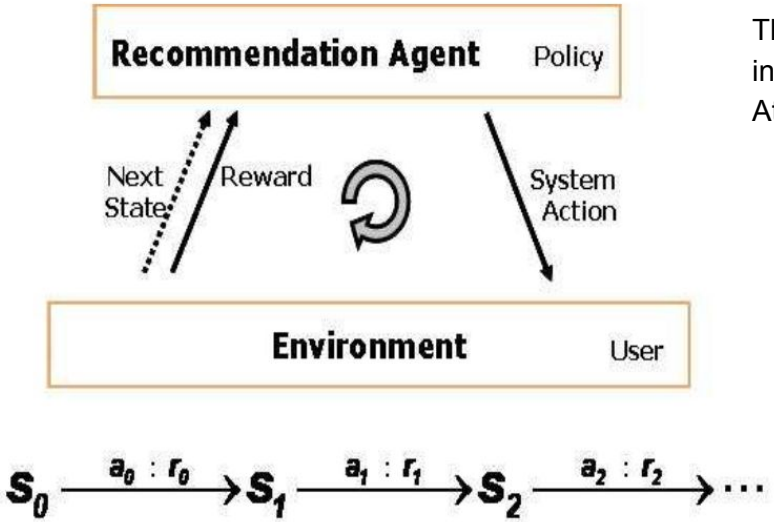


Figure 2: Interaction Model

The agent interacts with the user, who is part of its environment, in a series of interaction stages.
At each stage,

- the agent observes different aspects of the environment's state, and
- takes what it believes to be the best system action in that state, according to its policy;
- so, during the first stage, the agent receives a representation of the current state S0 and decides to take action a0.
- In response, the user then takes some action which the environment exploits in order to compute the next state S1, and also the reward r0 to the agent for action a0.
- The agent exploits r0 in order to learn to take better actions in the future.
- Then, the agent, after observing S1, takes action a1 (according to the policy) and receives a reward r1, along with the new state S2.
- Reinforcement Learning techniques guarantee that, as the agent-environment interaction session proceeds, the agent would eventually learn to take the best actions for all possible environment states, and would hence adopt the optimal policy.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

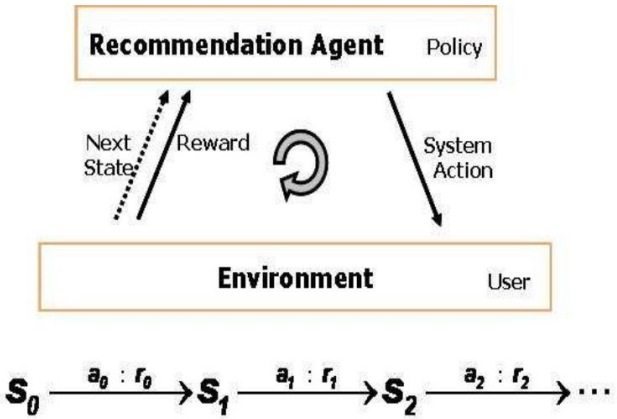


Figure 2: Interaction Model

Optimal policy actions are those which

- maximize the expected cumulative reward that the agent receives in the long run, i.e., at the end of the interaction session.

RL techniques guarantee that when the expected reward is maximized for each state of the environment, then the agent would converge to the optimal policy.

Although the interaction session will constitute a finite number of stages, we are not sure about their total count.

- In such a situation, the cumulative reward obtained by the agent during the session, RT , is normally computed with an *infinite-horizon discounted model*

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

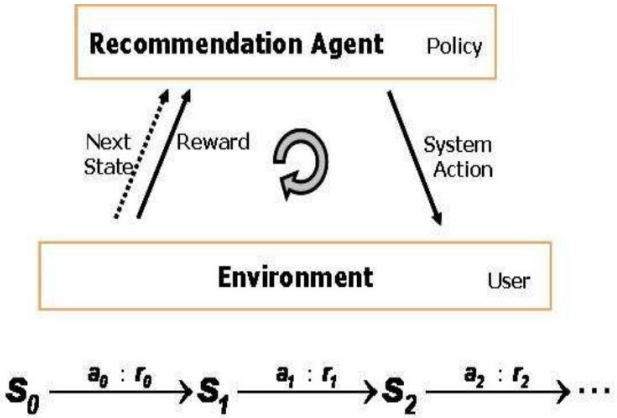


Figure 2: Interaction Model

Optimal policy actions are those which

- maximize the expected cumulative reward that the agent receives in the long run, i.e., at the end of the interaction session.

RL techniques guarantee that when the expected reward is maximized for each state of the environment, then the agent would converge to the optimal policy.

Although the interaction session will constitute a finite number of stages, we are not sure about their total count.

- In such a situation, the cumulative reward obtained by the agent during the session, RT , is normally computed with an *infinite-horizon discounted model*

$$R_T = E(\sum_{t=0}^{\infty} \gamma^t R_t)$$

where Rt is the reward obtained at the t-th stage and
γ is the discount rate parameter, $0 < \gamma \leq 1$.

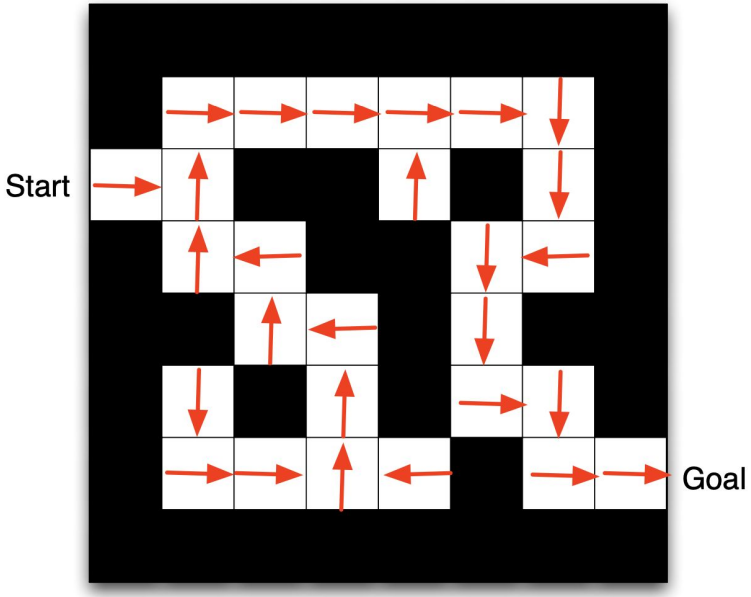
The discount rate determines the present value of the future rewards: a reward received after some stages (into the future) is likely to be of less worth, i.e., discounted, than if it is received immediately, i.e., during the current stage.
So, a large value of γ implies that the agent takes the future rewards into account very strongly while a smaller value implies that the agent aims to maximize just the immediate reward for the current stage.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Suppose that the agent is currently following some policy π. Then, we define the ***value*** of a state s under π, denoted by V π (s), as the

- expected cumulative reward which the agent receives when it starts in s and follows π thereafter.



[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

For MDPs, V π (s) satisfies the following recursive equation:

$$V^{\pi}(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} T(s, \pi(s), s') V^{\pi}(s') \qquad (1)$$

- where s′εS is the next state reached, when the agent takes action π(s) in s.
- The solution of equations yields the ***value function*** of the policy π, denoted by V π , which specifies the value for every state sεS .

The optimal behavior of the agent is given by a policy π∗ : S → A such that, for each initial state s, if the agent behaves accordingly to this policy, then RT is a maximum.
In this case, the expected reward RT for each states is called the ***optimal value*** for s,denoted by V ∗ (s): $V^*(s) = \max_{\pi} E(\sum_{t=0}^{\infty} \gamma^t R_t)$

This optimal value is unique and is the solution of the equations:

$$V^*(s) = \max_a (R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s')), \forall s \in S$$

Given the optimal value, the **optimal policy is given by:**

$$\pi^*(s) = arg\max_a (R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s'))$$

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Adaptive Recommendation Algorithm

Policy Iteration algorithm:

Policy Iteration requires an environment model in order to compute the optimal policy.

Given such a model and an initial arbitrary policy π, the agent (using this algorithm) iterates through the following two steps at each run:

- **Policy Evaluation:** where the agent computes the **value function** V π for the policy π by repeatedly solving the system of equations given in
- **Policy Improvement:** where the agent uses the computed value function, in order to **improve the policy** π.

Basically, for some state s, the algorithm determines whether the

- current action π(s) can be improved, i.e., whether some other action, say a, which the agent can take in s, is better than π(s).
- In fact, a would be better if the agent could accumulate more reward in s by executing a, rather than π(s).
- If the condition holds, then π is improved to take action a in s, i.e., π(s) = a.

This procedure is repeated for all the states in S. If any improvements occur in this phase, then a new run starts, i.e., the new policy is evaluated (first step) and then improved (second step).

This process continues until no improvement in the current policy is possible. Then, this current policy would be the optimal policy.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Figure 3: General Travel Preferences

Combines

- Interactive Query Management technology
- with collaborative ranking
- in order to recommend travel products that are promoted by the regional tourism organization

Assists the user in building a personalized *travel plan*, i.e., a collection of diverse travel products and services,

- e.g., accommodation, sports activities, cultural activities etc. that the user selects during her conversation with the recommender.

NutKing supports a human-computer interaction that mimics a typical counselling session in a real travel agency

Initially general travel preference

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Figure 4: Query Tightening Suggestions

Query Tightening Process (QTP):

In the situation where too many products are retrieved

the system exploits a feature selection method in order to suggest to the user how to *tighten* the current query,

- i.e., reduce the size of its retrieval set,
- or its result size.

To this end, it suggests three features from amongst those that are as yet unconstrained in the current query.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

Figure 5: Tightening Accepted

The system suggests the following features to the user: “cost/day of the hotel”, “hotel category” e.g. 3-star, 4-star etc, and “TV” i.e. whether the user would like to have a TV in his room.

If the user accepts the features “cost” and “TV” and constrains them in the query, then a small result size of 5 products is produced which the system displays to the user.

Suggest features for tightening when the result size is larger than some threshold value.

Otherwise, it executes the query and shows the list of retrieved products

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

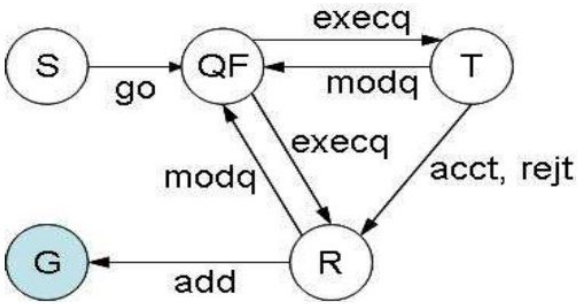


Figure 6: User Interaction Graph

In the start page (S), which is shown to the user each time he logs on to the system the user can only decide to continue by selecting the go function.

This always causes a transition to the page Query Form (QF) in which the user formulates, modifies and executes his queries.

If the user executes a query (execq) in QF , the system can transit to either

- the tightening page (T) or the result set page (R).

MDP describes the actions of the system and not the actions of the user.

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

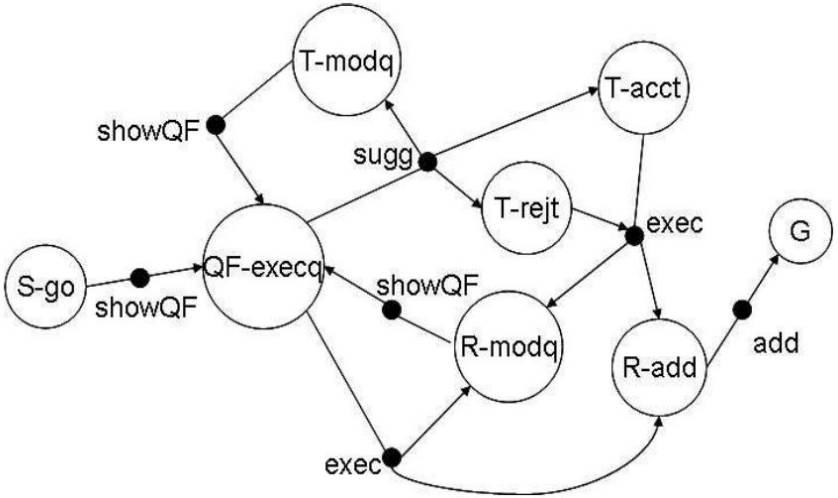


Figure 7: State Transition Diagram

The actions of the user and the page where the action is performed are considered as part of the state definition (see below for the state representation of QTP).

In the tightening page (T) the user can either accept one of the three proposed features for tightening (acct) or reject the suggestion (rejt).

Both of these user actions will lead to the result page (R), but with different results. In fact, if the user rejected tightening, then the original query is executed, and if the user accepted the tightening and provided a feature value then this user modified query is executed by the system.

Finally, in the result set page (R) the user can either add a product to the cart (add) or go back to the query form (QF) page, while requesting to modify the current query (modq).

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.4 Learning and Adaptability in Interactive Recommender System

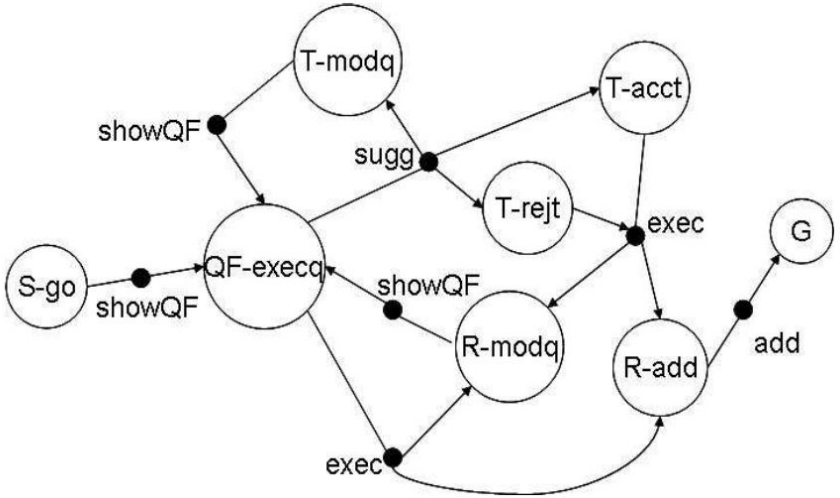


Figure 7: State Transition Diagram

The user behavior model must tell how the simulated user will behave during the simulated interaction in the following three cases:

- (Case 1) When the user is in state *QF-execq*, how will she modify the current query.
- (Case2) When the system suggests tightening(sugg),whether the user will decide to modify the query (*T-modq*), or to accept the tightening (*T-acct*), or to reject the tightening (*T- rejt*).
- (Case 3) When the system execute a query (exec), whether the user will add the test item to the cart (*R-add*) or modify the query (*R-modq*).

[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

2.1 Conversational Recommender System

A personalized conversational sales agent could have much commercial potential. E-commerce companies such as Amazon, eBay, JD, Alibaba etc. are piloting such kind of agent with their customers. However, the research on this topic is very limited and existing solutions are either based on single round adhoc search engine or traditional multi round dialog system. They usually only utilize user inputs in the current session, ignoring users' long term preferences. On the other hand, it is well known that sales conversion rate can be greatly improved based on recommender systems, which learn user preferences based on past purchasing behavior and optimize business oriented metrics such as conversion rate or expected revenue. In this work, we propose to integrate research in dialog systems and recommender systems into a novel and unified deep reinforcement learning framework to build a personalized conversational recommendation agent that optimizes a per session based utility function.

In particular, we propose to represent a user conversation history as a semi-structured user query with facet-value pairs. This query is generated and updated by belief tracker that analyzes natural language utterances of user at each step. We propose a set of machine actions tailored for recommendation agents and train a deep policy network to decide which action (i.e. asking for the value of a facet or making a recommendation) the agent should take at each step. We train a personalized recommendation model that uses both the user's past ratings and user query collected in the current conversational session when making rating predictions and generating recommendations. Such a conversational system often tries to collect user preferences by asking questions. Once enough user preference is collected, it makes personalized recommendations to the user. We perform both simulation experiments and real online user studies to demonstrate the effectiveness of the proposed framework.

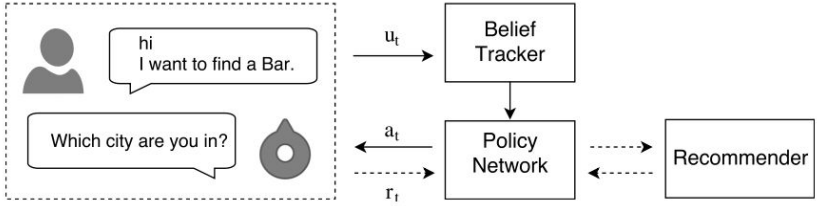


Figure 1: The conversational recommender system overview

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

Personalised conversational agents: Amazon, Alibaba

- High Commercial Impact

Existing solutions are based on:

- single round adhoc search engine or traditional multi round dialog system.
- only utilize user inputs in the current session, ignoring users' long term preferences.

Goal: Improve sales conversion rate by

- learning user preferences based on past purchasing behavior and
- optimize business oriented metrics such as conversion rate or expected revenue.

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

Unified deep reinforcement learning framework

- combine
  - the ranking and personalization ability of recommender system with
  - the sequential decision making power of the RL models
- build a personalized conversational recommendation agent that optimizes a per session based utility function

Goal:

- Analyze what the user has said in the current session
- interactively ask user clarification questions, and
- make personalized recommendations when appropriate, based on the
  - current session and
  - what the user has consumed or rated before

To achieve:

- how to understand the user's intention correctly
- how to make sequential decisions and take appropriate actions in each turn
- how to make personalized recommendations in order to maximize the user satisfaction

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

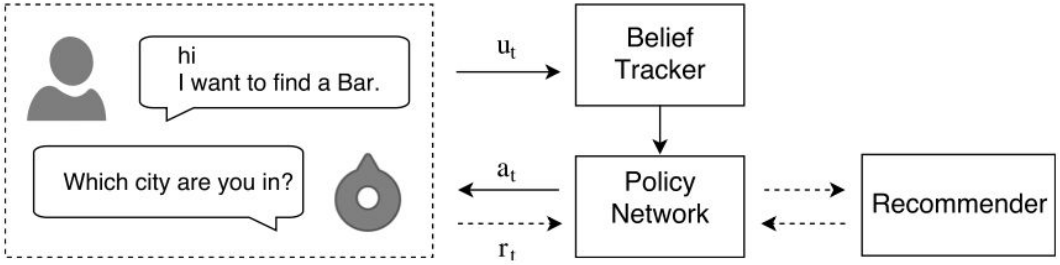


Figure 1: The Conversational Recommender System overview

Three components:

- Belief Tracker,
- Policy Network and
- Recommender

- At a time step in the dialogue, the user utters “I want to find a Bar”.
- The framework calls the belief tracker to convert the utterance into a vector representation or “belief”;
  - then the belief is sent to the policy network to make a decision.
  - For example, the policy network may decide to request the city information next.
- Then the agent may respond with “Which city are you in?”, and gets a reward, which is used to train the policy.
- For making recommendation:
  - the agent calls the recommender system to get a list of items personalized for the user.

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

Unified deep reinforcement learning framework to build a personalized conversational recommendation agent that optimizes a per session based utility function

- represent a user conversation history as a semi-structured user query with facet-value pairs.
- this query is generated and updated by belief tracker that analyzes natural language utterances of user at each step
- propose a set of machine actions tailored for recommendation agents and
- train a deep policy network to decide which action (i.e. asking for the value of a facet or making a recommendation) the agent should take at each step.
- Train a personalized recommendation model that uses both the user's past ratings and user query collected in the current conversational session when making rating predictions and generating recommendations.
- Such a conversational system often tries to collect user preferences by asking questions.
- Once enough user preference is collected, it makes personalized recommendations to the user.

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

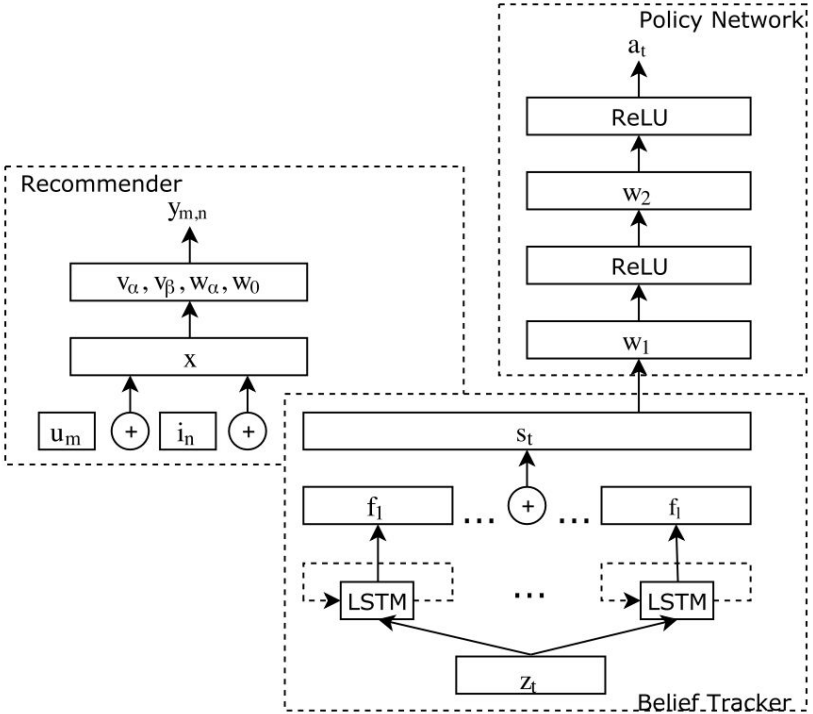


Figure 2: The structure of the proposed conversational recommender model:

Belief Tracker:

System understands which values the user has provided for product facets, and represents the user utterances with a semi-structured query

Extract facet-value pairs from user utterances during the conversation, and maintain the facet-value pairs as the memory state (i.e. user query) of the agent

Product facet (or attribute, metadata) f along with its specific value v as a facet-value pair (f ,v)

Each facet-value pair represents a constraint on the items. For example, (color,red) is a facet-value pair which constrains that the items need to be red in color.

The belief tracker takes the current and the past user utterances as the input, Outputs a probability distribution across all the possible values of a facet at the current time point

- et : user utterances, zt : Sequence of n-grams up to the current time
- ht : LSTM encodes zi ft: softmax(hi)
- st : f_i are concatenated to each other to form the agent's current belief of the dialogue state in the current session

$$\mathbf{z}_t = ngram(e_t) \quad (1)$$

$$\mathbf{h}_t = LSTM(\mathbf{z}_1, \mathbf{z}_2, ..., \mathbf{z}_t) \quad (2)$$

$$\mathbf{f}_i = softmax(\mathbf{h}_t) \quad (3)$$

$$s_t = \mathbf{f}_1 \oplus \mathbf{f}_2 ... \oplus \mathbf{f}_l \quad (4)$$

2.1 Conversational Recommender System

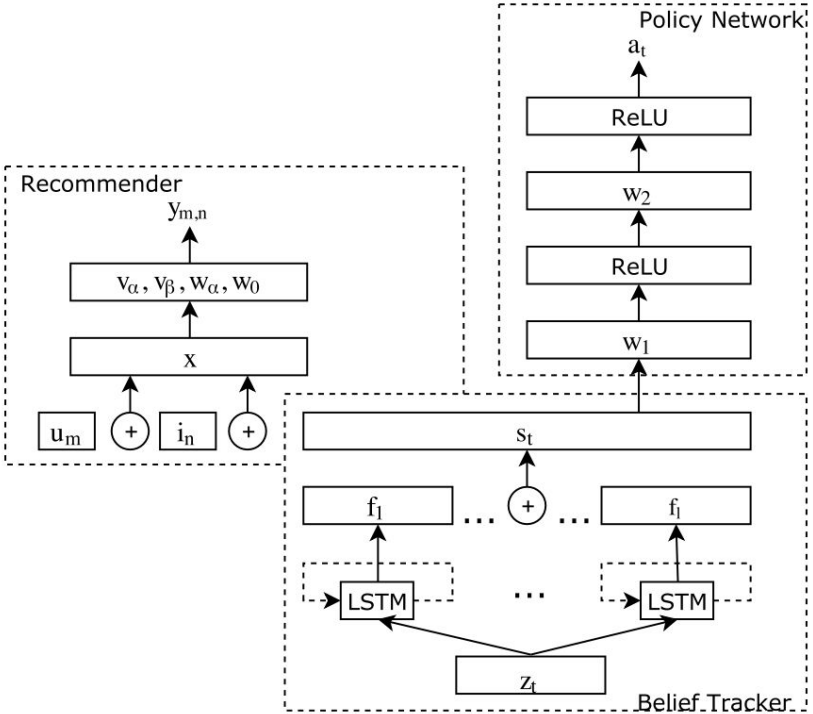


Figure 2: The structure of the proposed conversational recommender model:

Recommender:

As the conversational system interacts with the users,

- at certain round, the conversational system can decide to make a recommendation
- based on its current belief of the user's information need,
- which is interpreted as the dialogue state.

Train the recommender with the dialogue state, user information and item information.

Use Factorization Machine (FM), for the reason that FM can combine different features, e.g. s_t , together to train the recommendation model.

[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018

2.1 Conversational Recommender System

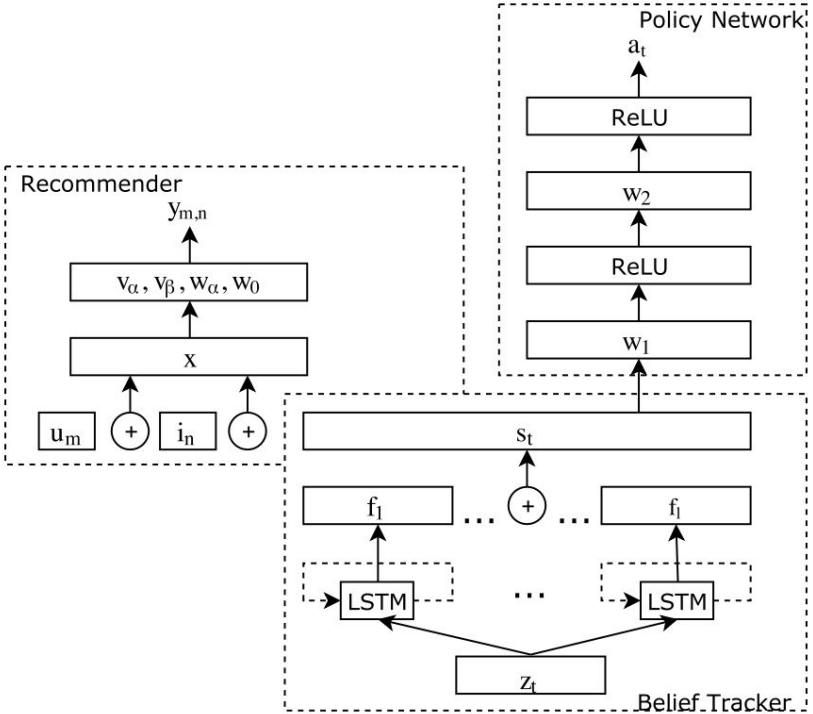


Figure 2: The structure of the proposed conversational recommender model:

Deep Policy Network (1/2):

At each turn, the RL model selects an action based on the dialogue state, in order to maximize the long term return

- learn policy directly (without consulting value function)

State: The state s_t is the current description of the environment from the viewpoint of the agent.

- description of the conversation context, which is the belief tracker's output, $s_t = \{f_1 \oplus f_2 ... \oplus f_l\}$.

Action: An action a_t is the decision the agent needs to make at time step t . Two kinds of actions

- request the value of a facet, which is further divided into l actions $\{a_1, a_2, ..., a_l\}$, one per each facet.
- make a personalized recommendation a_{rec}, in which case the recommendation module would be called.

Reward: The reward is the benefit/penalty the agent gets from interacting with its environment.

At each turn, according to the current state s_t , the agent selects an action a_t following the policy, and it gets an immediate reward r_t , denoting how good the current decision is.

- The state s_t transits to a new state s'.

The conversational system gets a reward when it requests a facet value, or makes a recommendation.

2.1 Conversational Recommender System

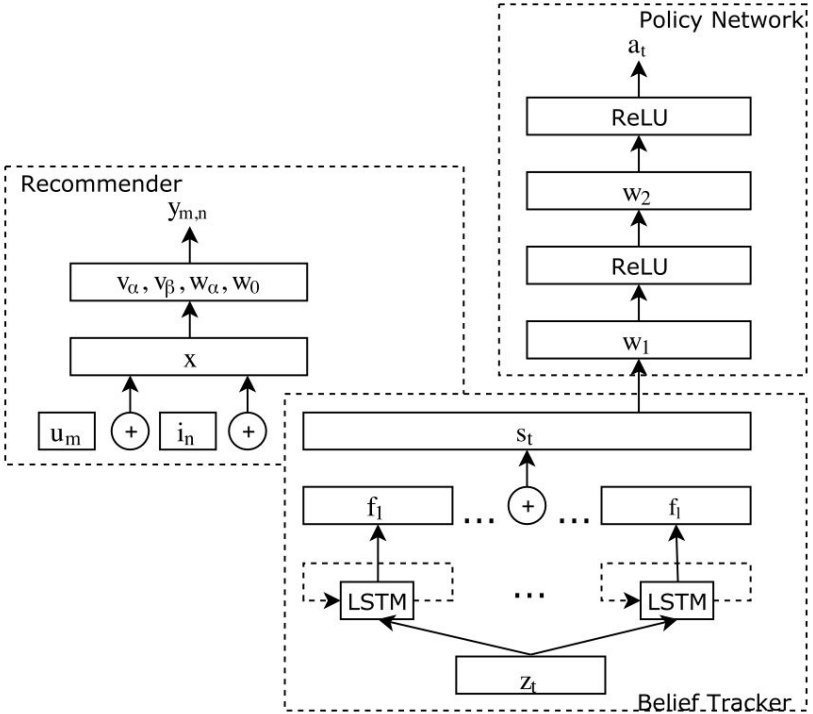


Figure 2: The structure of the proposed conversational recommender model:

Deep Policy Network (2/2):

Policy: This is the target the model tries to learn. Usually denoted as $\pi(a_t|s_t)$,

- the policy represents the score, such as the probability, of taking action a_t when the agent is in state s_t.

For simplicity and without loss of generality,

- two fully connected layers as the policy network,
- each layer with a ReLU activation function.

The output of the network is further sent to a softmax layer.

Specifically, the goal of the policy network is to maximize the episodic expected reward from the starting state:

$$\eta(\theta) = E_\pi[\sum_{t=0}^{T} \gamma^t r_t] \quad (10)$$

where θ is the policy parameter to be learned,
γ is a discount parameter emphasizing more on the immediate reward than the future rewards, and
r_t is the reward at time step t.

REINFORCE algorithm, the gradient of the learning object becomes:

$$\nabla\eta(\theta) = E_\pi[\gamma^t G_t \nabla_\theta \log \pi(a_t|s_t, \theta)] \quad (11)$$

G_t is the sum of rewards, or return, starting from time step t to the final time step T.

- we always have a terminating state of the conversation, e.g., the **user leaves** the chat or the user is **successfully recommended** with a target.
- Each such kind of sequence is called an episode in RL

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Recommender systems are embracing conversational technologies to obtain user preferences dynamically, and to overcome inherent limitations of their static models. A successful Conversational Recommender System (CRS) requires proper handling of interactions between conversation and recommendation. We argue that three fundamental problems need to be solved: 1) what questions to ask regarding item attributes, 2) when to recommend items, and 3) how to adapt to the users' online feedback. To the best of our knowledge, there lacks a unified framework that addresses these problems.

In this work, we fill this missing interaction framework gap by proposing a new CRS framework named Estimation–Action– Reflection, or EAR, which consists of three stages to better converse with users. (1) Estimation, which builds predictive models to estimate user preference on both items and item attributes; (2) Action, which learns a dialogue policy to determine whether to ask attributes or recommend items, based on Estimation stage and conversation history; and (3) Reflection, which updates the recommender model when a user rejects the recommendations made by the Action stage. We present two conversation scenarios on binary and enumerated questions, and conduct extensive experiments on two datasets from Yelp and LastFM, for each scenario, respectively. Our experiments demonstrate significant improvements over the state-of-the-art method CRM [32], corresponding to fewer conversation turns and a higher level of recommendation hits.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Recommender systems are

- embracing conversational technologies to obtain user preferences dynamically, and
- to overcome inherent limitations of their static models.

A successful Conversational Recommender System (CRS)

- requires proper handling of interactions between conversation and recommendation.

We argue that three fundamental problems need to be solved:

1. What questions to ask regarding item attributes,
2. When to recommend items, and
3. How to adapt to the users' online feedback

Proposed CRS framework named Estimation–Action– Reflection, or EAR, which consists of three stages to better converse with users.

1. **Estimation**, which builds predictive models to estimate user preference on both items and item attributes;
2. **Action**, which learns a dialogue policy to determine whether to ask attributes or recommend items, based on Estimation stage and conversation history; and
3. **Reflection**, which updates the recommender model when a user rejects the recommendations made by the Action stage.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

In multi-round setting CRS needs to strategically plan its actions.
The key in performing such planning, lies in the interaction between

- Conversational Component (CC; responsible for interacting with the user) and
- Recommender Component (RC; responsible for estimating user preference – e.g., generating the recommendation list).

Three fundamental problems toward the deep interaction between CC and RC as follows:

1. What attributes to ask? A CRS needs to choose which attribute to ask the user about.
To achieve the goal of hitting the right items in fewer turns,

- the CC must consider whether the user will like the asked attribute.
- This is exactly the job of the RC which scrutinizes the user's historical behavior.

2. When to recommend items? With sufficient certainty, the CC should push the recommendations generated by the RC.
A good timing to push recommendations should be when

- the candidate space is small enough
- asking additional questions is determined to be less useful or helpful, from the perspective of either information gain or user patience; and
- the RC is confident that the top recommendations will be accepted by the user.
- Determining the appropriate timing should take both the conversation history of the CC and the preference estimation of the RC into account.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

3. How to adapt to users' online feedback? After each turn, the user gives feedback; i.e., "yes"/"no" to a queried attribute, or an "accept"/"reject" the recommended items.

- For "yes" on the attribute, both user profile and item candidates need to be updated to generate better recommendations; this requires the offline RC training to take such updates into account.
- For "no', the CC needs to adjust its strategy accordingly.
- If the recommended items are rejected, the RC model needs to be updated to incorporate such a negative signal. Although adjustments seem only to impact either the RC or the CC, such actions impact both.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Estimation–Action–Reflection (EAR), which consists of three stages.

1. Estimation: which builds predictive models offline to estimate user preference on items and item attributes.
Train a factorization machine (FM) using user profiles and item attributes as input features.

- the joint optimization of FM on the two tasks of item prediction and attribute prediction, and
- the adaptive training of conversation data with online user feedback on attributes.

2. Action: which learns the conversational strategy that

- determines whether to ask or recommend, and what attribute to ask.
- train a policy network with reinforcement learning, optimizing the reward of shorter turns and
- successful recommendations based on the FM's estimation of user preferred items and attributes, and the dialogue history.

3. Reflection: which adapts the CRS with user's online feedback.

- specifically, when a user rejects the recommended items, construct new training triplets by treating the items as negative instances and update the FM in an online manner

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

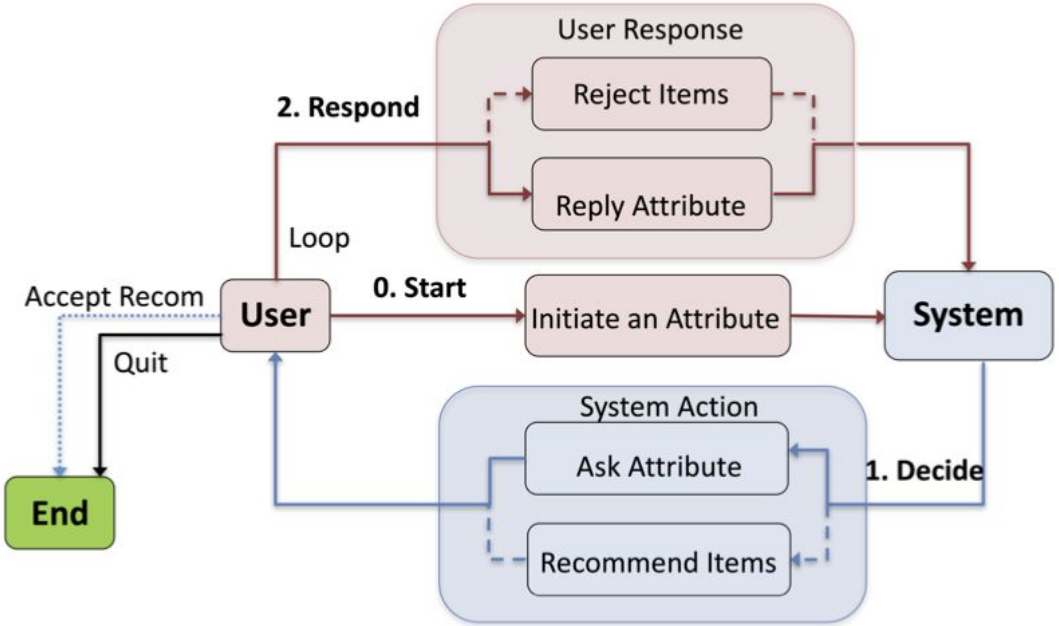


Figure 1: The workflow of our multi-round conversational recommendation scenario. The system may recommend items multiple times, and the conversation ends only if the user accepts the recommendation or chooses to quit.

Actions:

- Ask
- Recommend

CRS may ask zero to many questions before making recommendations.

A session terminates if a user accepts the recommendations or leaves due to his impatience.

Goal of the CRS as making desired recommendations within as few rounds as possible.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Estimation:

1. Basic Recommendation Method. Use the factorization machine (FM) as predictive model due to its success and wide usage in recommendation tasks.

FM considers all pairwise interactions between input features, which is costly and may introduce undesired interactions that negatively affect

Thus, only keep the interactions that are useful to task and remove the others

To train the FM, we optimize the pairwise Bayesian Personalized Ranking (BPR)

2. Attribute-aware BPR for Item Prediction: The emphasis of CRS is to rank the items that contain the user preferred attributes well.

Critical for the model ranking well on the current candidates. This is very important for CRS since the candidate items dynamically change with user feedback along the conversation.

3. Attribute Preference Prediction: Accurate attribute prediction separately. This prediction of attribute preference is mainly used in the CC to support the action on which attribute to ask

4. Multi-task Training: We perform joint training on the two tasks of item prediction and attribute prediction, which has the potential of mutual benefits since their parameters are shared. The multi-task training objective is:

$$L = L_{item} + L_{attr}. \tag{9}$$

First train the model with Litem.

After it converges, continue optimizing the model using Lattr .

Iterate the two steps until convergence under both losses.

Empirically, 2-3 iterations are sufficient for convergence.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Action:
After the estimation stage, the action stage finds the best strategy for when to recommend.
Adopt reinforcement learning (RL) to tackle multi-round decision making problem, aiming to accomplish successful recommendation in shorter number of turns.
Use templates as wrappers to handle user utterances and system response generation.
This serves as an upper bound study of real applications as do not include the errors for language understanding and generation

1 State Vector: The state vector is a bridge for the interaction between the CC and RC.
Encode information from the RC and dialogue history into a state vector, providing it to the CC to choose actions.
The state vector is a concatenation of four component vectors that encode signal from different perspectives:

$$\mathbf{s} = \mathbf{s}_{ent} \oplus \mathbf{s}_{pre} \oplus \mathbf{s}_{his} \oplus \mathbf{s}_{len}. \quad (10)$$

Sent: This vector encodes the entropy information of each attribute among the attributes of the current candidate items Vcand . The intuition is that asking attributes with large entropy helps to reduce the candidate space, thus benefits finding desired items in fewer turns.

Spre: This vector encodes u's preference on each attribute
The intuition is that the attribute with high predicted preference is likely to receive positive feedback, which also helps to reduce the candidate space.

Shis: This vector encodes the conversation history
This state is useful to determine when to recommend items. For example, if the system has asked about a number of attributes for which u approves, it may be a good time to recommend.

Slen: This vector encodes the length of the current candidate list. The intuition is that if the candidate list is short enough, EAR should turn to recommending to avoid wasting more turns.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Action:

2. Policy Network and Rewards:

The conversation action is chosen by a policy network in our CC.

Choose a simple policy network — a two-layer multi-layer perceptron,

- which can be optimized with the standard policy gradient method.
- It contains two fully-connected layers and maps the state vector s into the action space.
- The output layer is normalized to be a probability distribution over all actions by so f tmax.

In terms of the action space, we includes all attributes P and a dedicated action for recommendation

After the CC takes an action at each turn, it will receive an immediate reward from the user (or user simulator).

This will guide the CC to learn the optimal policy that optimizes long-term reward.

Four kinds of rewards:

- **rsuc:** a strongly positive reward when the recommendation is successful
- **rask:** a positive reward when the user gives positive feedback on the asked attribute
- **rquit:** a strongly negative reward if the user quits the conversation
- **rprev:** a slightly negative reward on every turn to discourage overly lengthy conversations

The intermediate reward rt at turn t is the sum of the above four rewards:

$$r_t = r_{suc} + r_{ask} + r_{quit} + r_{prev}$$

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Policy Network π(at |st),which returns the probability of taking action at given the state st

Here at ∈ A and st denote the action to take and the state vector of the t-th turn, respectively.

To optimize the policy network, use the standard policy gradient method, formulated as follows:

$$\theta \leftarrow \theta - \alpha \nabla \log \pi_\theta(a^t \mid \mathbf{s}^t) R_t, \qquad (11)$$

θ denotes the parameter of the policy network,
α denotes the learning rate of the policy network, and

Rt is the total reward accumulating from turn t to the final turn T

$$R_t = \sum_{t'=t}^{T} \gamma^{T-t'} r_{t'}$$

where γ is a discount factor which discounts future rewards over immediate reward

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

Reflection:
This stage also implements the interaction between the CC and RC.
It is triggered when the CC pushes the recommended items Vt to the user but gets rejected,

- so as to update the RC model for better recommendations in future turns.

Rejection feedback is an explicit signal on user dislikes which are highly valuable to utilize

- it indicates that the offline learned FM model improperly assigns high scores to the rejected items

To leverage on this source of feedback, treat the rejected items Vt as negative samples, constructing more training examples to refresh the FM model

Following the offline training process, we also optimize the BPR loss:

$$L_{ref} = \sum_{(u,v,v')\in\mathcal{D}_4} -\ln\sigma(\hat{y}(u,v,\mathcal{P}_u)-\hat{y}(u,v',\mathcal{P}_u))+\lambda_\Theta \|\Theta\|^2 \quad (12)$$

where:
$\mathcal{D}_4 := \{(u,v,v') \mid v \in \mathcal{V}_u^+ \wedge v' \in \mathcal{V}^t\}$

This stage is performed in an online fashion, where we do not have access to the ground truth positive item.

Thus, treat the historically interacted items V+ as the positive items to pair with the rejected items

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

2.3 Estimation–Action–Reflection: Towards Deep Interaction Between CRS

We present two conversation scenarios on binary and enumerated questions, and conduct extensive experiments on two datasets from Yelp and LastFM, for each scenario, respectively. Our experiments demonstrate significant improvements over the state-of-the-art method CRM [32], corresponding to fewer conversation turns and a higher level of recommendation hits.

[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020

3 DRL approaches for CRS - II (20min)

- Towards Topic-Guided Conversational Recommender System [7]
- Interactive Path Reasoning on Graph for Conversational Recommendation [8]
- Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning [9]
- Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning [10]

3.1 Towards Topic-Guided Conversational Recommender System

Conversational recommender systems (CRS) aim to recommend high-quality items to users through interactive conversations. To develop an effective CRS, the support of high-quality datasets is essential. Existing CRS datasets mainly focus on immediate requests from users, while lack proactive guidance to the recommendation scenario. In this paper, we contribute a new CRS dataset named TG-ReDial (Recommendation through Topic-Guided Dialog). Our dataset has two major features. First, it incorporates topic threads to enforce natural semantic transitions towards the recommendation scenario. Second, it is created in a semi-automatic way, hence human annotation is more reasonable and controllable. Based on TG-ReDial, we present the task of topic-guided conversational recommendation, and propose an effective approach to this task. Extensive experiments have demonstrated the effectiveness of our approach on three sub-tasks, namely topic prediction, item recommendation and response generation. TG-ReDial is available at https://github.com/RUCAIBox/TG-ReDial.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

Conversational recommender systems (CRS) aim to

- recommend high-quality items to users through interactive conversations.

To develop an effective CRS, the support of high-quality datasets is essential.

- Existing CRS datasets mainly focus on immediate requests from users, while lack proactive guidance to the recommendation scenario.

In this paper, we contribute a new CRS dataset named TG-ReDial (Recommendation through Topic-Guided Dialog).

Dataset has two major features.

First, it incorporates topic threads to enforce natural semantic transitions towards the recommendation scenario.

Second, it is created in a semi-automatic way, hence human annotation is more reasonable and controllable.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

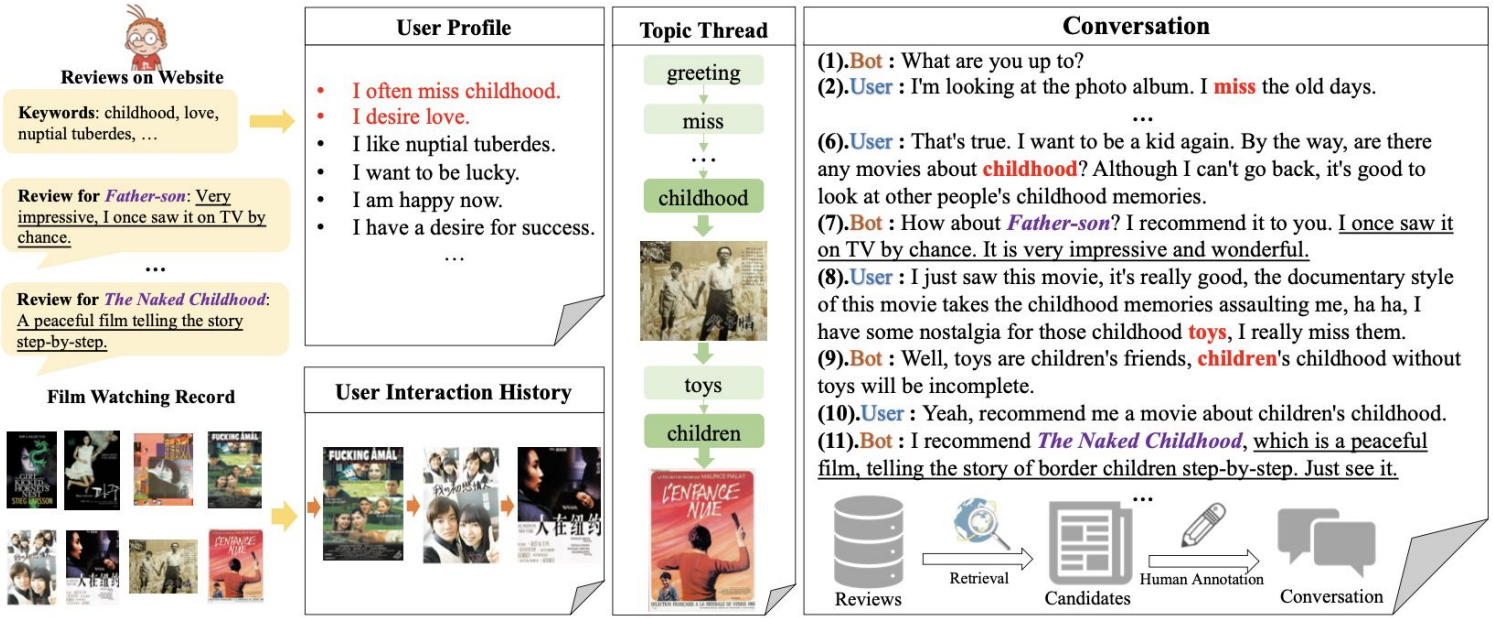


Figure 1: An illustrative example for TG-ReDial dataset. We utilize real data to construct the recommended movies, topic threads, user profiles and utterances. Other user-related information (*e.g.,* historical interaction records) is also available in our dataset.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

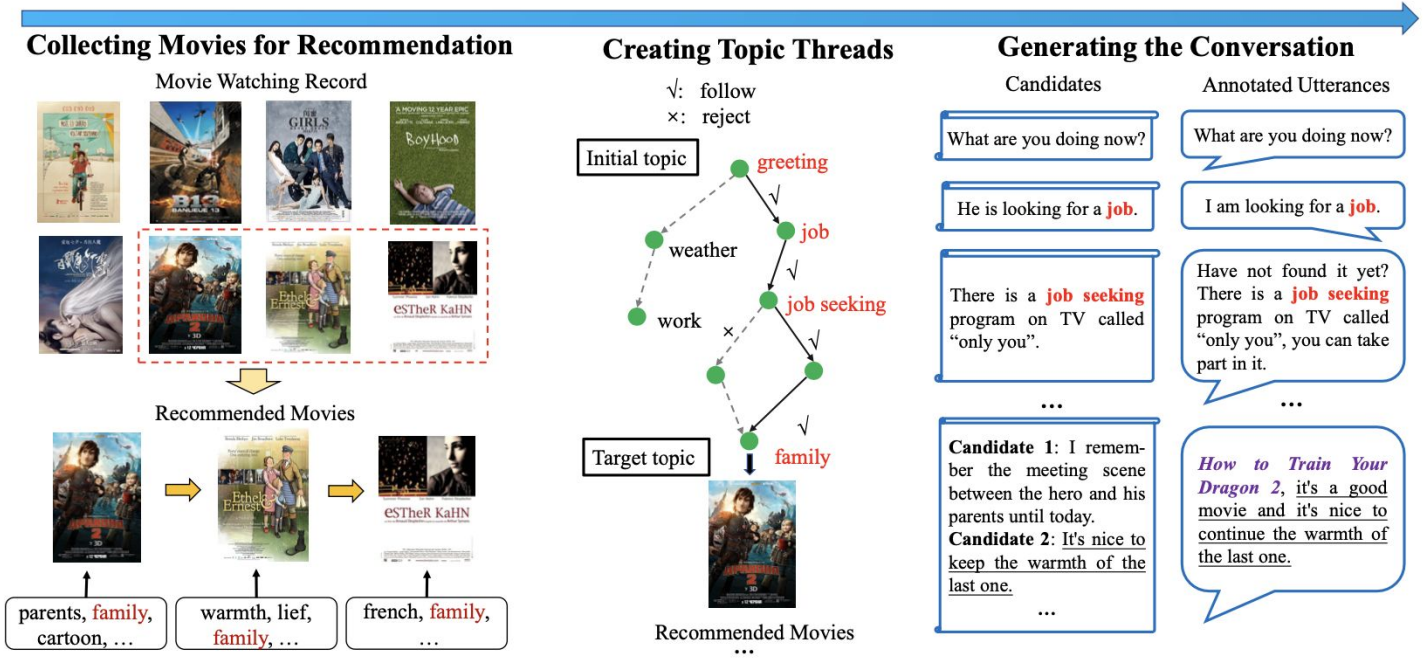


Figure 2: An example of the data collection process of TG-ReDial. We select three movies sharing the same tag of "*family*" from film watching record for recommendation, then create the topic thread (marked in red font) on ConceptNet, and finally provide high-quality candidate sentences for human annotation.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

Problem Formulation:

Given a user u, we assume that she/he is associated with

- a profile Pu (a set of descriptive sentences related to the topics that u is interested in) and
- a historical interaction sequence Iu (a chronologically- ordered sequence of items that u has interacted with)

Given

- the user profile P_u, user interaction sequence I_u,
- historical utterances $\{s_1, \ldots, s_{k-1}\}$ and
- corresponding topic sequence $\{t_1, \ldots, t_{k-1}\}$,

Goal:

- predict the next topic t_k to reach the target topic, (topic prediction)
- recommend the movie i_k, and finally (item recommendation)
- produce a proper response s_k about the topic or with persuasive reason (response generation)

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

Recommendation Module:
Predict the item that a user likes given the conversation context.
The key point is how to derive an effective user presentation for recommendation

Utilize

- the pre-trained language model BERT to encode the historical utterances $\{s_1, \ldots, s_{k-1}\}$, and
- a self-attentive sequential recommendation model SASRec to encode the user interaction sequence I_u.
- This gives representation v_u of user u

Given the user representation,compute the probability that recommends an item i from the item set to a user u:

$$\mathrm{P}_{rec}(i) = \mathrm{softmax}(\boldsymbol{e}_i^{\top} \cdot \boldsymbol{v}_u) \qquad (2)$$

where ei is the learned item embedding for item i.

Utilize Equation 2 to rank all the items and select the item with the largest probability for recommendation.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

Dialog Module:
Goal: Generate proper responses to the user (seeker) for topic guidance or item recommendation

Topic Prediction Model: Predicts the next topic that guides user u towards the target topic.
Utilize text data for topic prediction, and implement three different BERT-based encoders, namely

- conversation-BERT - for encoding the historical utterances
- topic-BERT - historical topic sequence
- profile-BERT - user profile

For each BERT variant, simply concatenate all the available text data and the target topic (with [SEP] tokens for separating).
The incorporation of target topic is to enhance the topical semantics.
Based on the obtained representations, compute the probability of a topic t as the next topic by:

$$P_{topic}(t) = \text{softmax}(e_t^\top \cdot \text{MLP}([r_{(1)}; r_{(2)}; r_{(3)}]), \quad (3)$$

where e_t is the learned embedding for topic t, $r_{(1)}$, $r_{(2)}$ and $r_{(3)}$ are the embeddings of historical utterances, topics and user profile obtained from conversation-BERT, topic-BERT and profile-BERT, respectively.

Utilize Eq. 3 to rank all the topics and select the topic with the largest probability.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

Response Generation Model:
Goal: Generate proper responses for topic-guided conversations.

Leverage the pre-trained text generation model GPT-2 for response generation.

- GPT-2 utilizes a stacked of masked multi-head self-attention layers trained on massive web-text data by the generic language model

For non-recommendation case: generate the response conditioned on the predicted topic, and concatenate t_k with the historical utterances $\{s_1, \ldots, s_{k-1}\}$ (separated by [SEP] tokens).

For recommendation case: generate the persuasive reason conditioned on the recommended item, and concatenate the recommended movie i_k with the historical utterances $\{s_1, \ldots, s_{k-1}\}$.

For both cases, we can unify the input as a long sequence, which will be encoded and fed into GPT-2 for decoding.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.1 Towards Topic-Guided Conversational Recommender System

	Total		Average
# Users	1,482	# Words per Utterance	19.0
# Dialogues	10,000	# Topics per Dialogue	7.9
# Utterances	129,392	# Movies per Dialogue	3
# Movies	33,834	# Profile Sentences per User	10
# Topics	2,571	# Watched Movies per User	202.7

Table 1: Data statistics of our TG-ReDial dataset.

Based on TG-ReDial, we present the task of topic-guided conversational recommendation, and propose an effective approach to this task. Extensive experiments have demonstrated the effectiveness of our approach on three sub-tasks, namely topic prediction, item recommendation and response generation. TG-ReDial is available at https://github.com/RUCAIBox/TG-ReDial.

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

Traditional recommendation systems estimate user preference on items from past interaction history, thus suffering from the limitation of obtaining fine-grained and dynamic user preference. Conversational recommendation system (CRS) brings new solutions to overcome these limitations by enabling the system to directly ask users about their preferred attributes on items. However, existing CRS methods do not make full use of such advantage — they only use the attribute feedback in a rather implicit way to help update the latent user representation. In this paper, we propose Conversational Path Reasoning (CPR), a generic framework that models conversational recommendation as an interactive path reasoning problem on a graph. It walks through the attribute vertices by following user feedback, utilizing the user preferred attributes in an explicit way. By leveraging on the graph structure, CPR is able to prune off many irrelevant candidate attributes, leading to better chance of hitting user preferred attributes. To demonstrate how CPR works, we propose a simple yet effective instantiation named SCPR (Simple CPR). We perform empirical studies on the multi-round conversational recommendation scenario, the most realistic CRS setting so far that considers multiple rounds of asking attributes and recommending items. Through extensive experiments on two datasets Yelp and LastFM, we validate the effectiveness of our SCPR, which significantly outperforms the state-of-the-art CRS methods EAR [13] and CRM [24]. In particular, we find that the more attributes there are, the more advantages our method can achieve.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

Traditional recommendation systems

- estimate user preference on items from past interaction history,
- thus suffering from the limitations of obtaining fine-grained and dynamic user preference.

Conversational recommendation system (CRS) brings revolutions to those limitations by

- enabling the system to directly ask users about their preferred attributes on items.

However, existing CRS methods do not make full use of such advantage — they only use the attribute feedback in rather implicit ways such as updating the latent user representation.

Propose Conversational Path Reasoning (CPR),

- a generic framework that models conversational recommendation as an interactive path reasoning problem on a graph.
- It walks through the attribute vertices by following user feedback, utilizing the user preferred attributes in an explicit way.
- By leveraging on the graph structure, CPR is able to prune off many irrelevant candidate attributes, leading to better chance of hitting user preferred attributes.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

1 Crystally explainable:

- It models conversational recommendation as an interactive path reasoning problem on the graph, with each step confirmed by the user.
- Thus, the resultant path is the correct reason for the recommendation.
- This makes better use of the fine-grained attribute preference than existing methods that only model attribute preference in latent space

2 Exploitation:

- It facilitates the exploitation of the abundant information by introducing the graph structure.
- By limiting the candidate attributes to ask as adjacent attributes of the current vertex, the candidate space is largely reduced,
- leading to a significant advantage compared with existing CRS methods like that treat almost all attributes as the candidates.

3. Natural combination and mutual promotion of conversation system and recommendation system:

- the path walking over the graph provides a natural dialogue state tracking for conversation system, and
- it is believed to be efficient to make the conversation more logically coherent
- on the other hand, being able to directly solicit attribute feedback from the user,
    - the conversation provides a shortcut to prune off searching branches in the graph.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

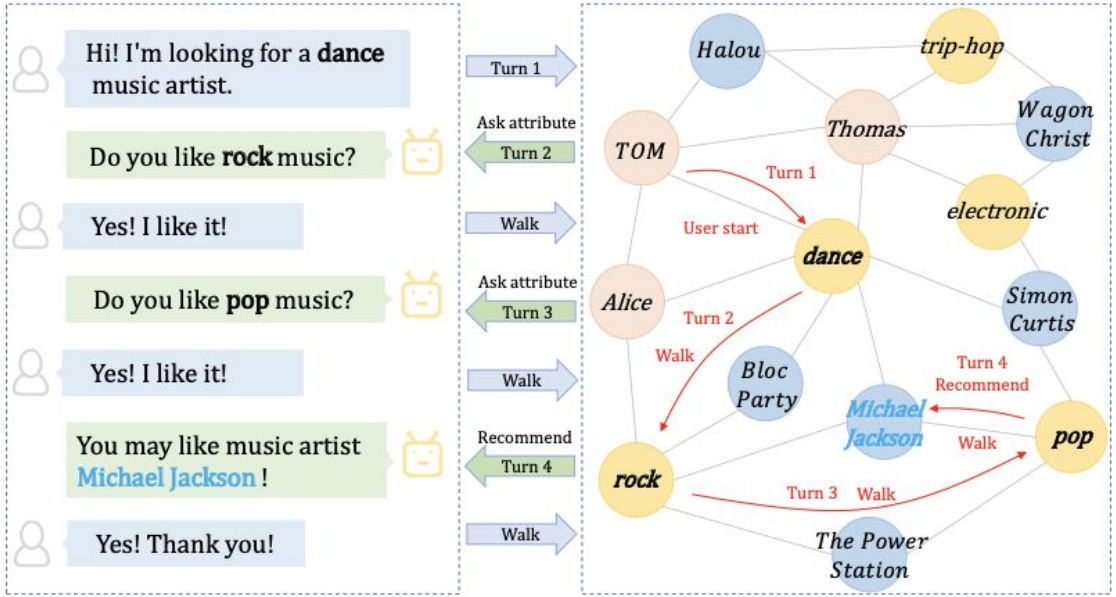
3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

u, v, p	User, item, and attribute
P	An *active* attribute path in the graph
aa_t	An adjacent attribute of the attribute p_t
$\mathcal{AA}_t$	The set of adjacent attributes of the attribute p_t
$\mathcal{P}_u$	The set of attributes confirmed by u in a session
$\mathcal{P}_{cand}$	The set of candidate attributes
$\mathcal{V}_p$	The set of items that contain the attribute p
$\mathcal{V}_{cand}$	The set of candidate items;
a	The action of CPR, either a_{ask} or a_{rec}

Table 1: Main notations used in the paper

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation



Light orange: User

Light blue: Attribute

Light gold: Items

The artist Michael Jackson is an item and

- the attributes are rock, dance etc.

Figure 1: An illustration of interactive path reasoning in CPR.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

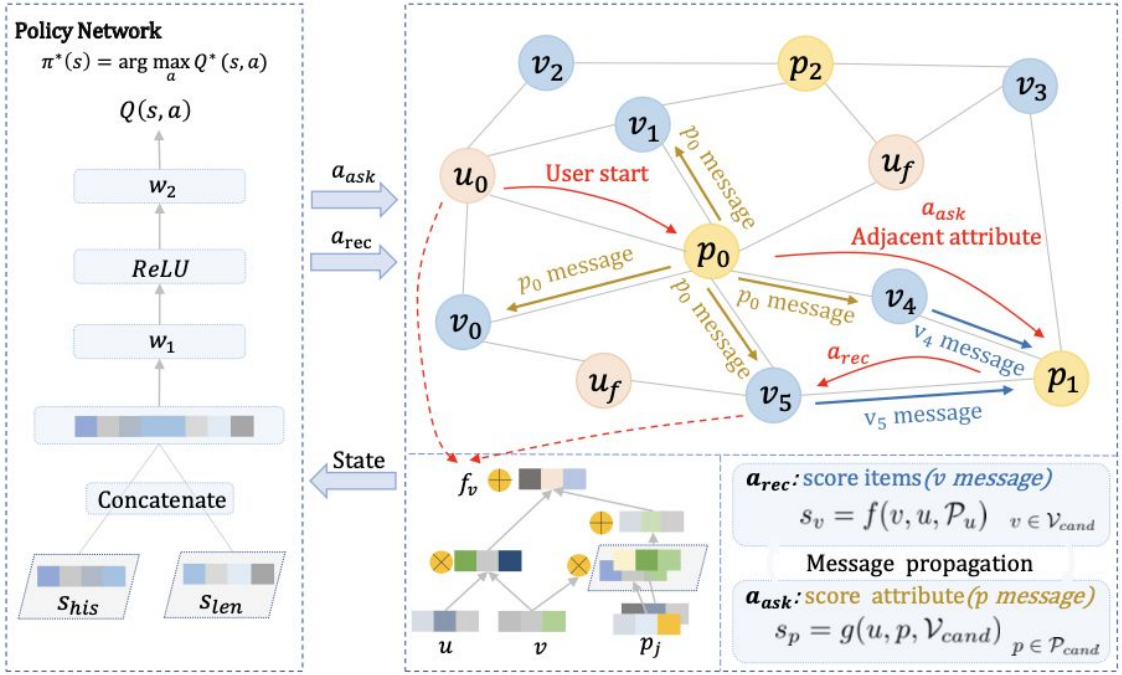


Figure 2: CPR framework overview.

It starts from the user u0

walks over adjacent attributes,

- forming a path (the red arrows) and
- eventually leading to the desired item.

The policy network (left side) determines whether to

- ask an attribute or
- recommend items in a turn.
-

Reasoning functions

- f score attributes
- д score tems

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

Consultation: Once a reasoning step is completed,CPR moves to the consultation step. The purpose of this step is to decide whether

- to ask an attribute or to recommend items,
- with the goal of achieving successful recommendations in fewest turns.

A policy function π (s) is expected to make the decision

- based on the global dialogue state s,
- which can include any information useful for successful recommendation, such as the dialogue history, the information of candidate items.
- The output action space of the policy function contains two choices:a-ask or a-rec,
- Indicating whether to perform top-k recommendations or to ask an attribute in this turn.
- If the RL decision is a-ask ,
    - directly take highest-scored attribute from Pcand based on score sp
    - otherwise, recommend top-k items from Vcand based on score sv

Design of RL here reflects another major difference with existing conversational recommendation systems EAR and CRM.
Although they also learn policy networks with RL, their policy is to

- decide which attribute to ask, rather than our choice of whether to ask attribute.
- Which means, the size of their action space is |P| + 1, where |P| denotes the number of attributes.
- This greatly increases the difficulty to learn the policy well, especially for a large |P|, since RL is notoriously difficult to train when the action space is large.
- In contrast, the action space of our RL is of size 2, being much easier to train.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

Consultation - RL Policy:

- Use a two-layer feed forward neural network as policy network.
- For the ease of convergence, use the standard Deep Q-learning for optimization

The policy network takes the state vector s as input and outputs the values Q(s,a) for the two actions, indicating the estimated reward for a-ask or a-rec.

A system will always choose the action with higher estimated reward. The state vector s is a concatenation of two vectors:

$$\mathbf{s} = \mathbf{s}_{his} \oplus \mathbf{s}_{len},$$

Shis encodes the conversation history, which is expected to guide the system to act smarter,

- e.g., if the asked attributes are accepted for multiple turns, it might be a suitable timing to recommend.

Slen encodes the size of candidate item set. As discussed by,

- it is easier to make successful recommendations when there are fewer candidate items.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

Rewards: Five kinds of rewards
(1) rrec_suc, a strongly positive reward when the recommendation succeeds,
(2) rrec_f ail , a strongly negative reward when the recommendation fails,
(3) rask_suc , a slightly positive reward when the user accepts an asked attribute,
(4) rask_f ail , a negative reward when the user rejects an asked attribute, and
(5) rquit , a strongly negative reward if the session reaches the maximum number of turns.

The accumulated reward is the weighted sum of these five.

The detailed rewards to train the policy network are:

$$rrec\_suc=1, rrec\_f\ ail=-0.1, rask\_suc=0.01, rask\_f\ ail=-0.1, rquit=-\ 0.3$$

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.2 Interactive Path Reasoning on Graph for Conversational Recommendation

To demonstrate how CPR works, we propose a simple yet effective instantiation named SCPR (Simple CPR). We perform empirical studies on the multi-round conversational recommendation scenario, the most realistic CRS setting so far that considers multiple rounds of asking attributes and recommending items. Through extensive experiments on two datasets Yelp and LastFM, we validate the effectiveness of our SCPR, which significantly outperforms the state-of-the-art CRS methods EAR [13] and CRM [24]. In particular, we find that the more attributes there are, the more advantages our method can achieve.

[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

Recommender systems play a crucial role in mitigating the problem of information overload by suggesting users' personalized items or services. The vast majority of traditional recommender systems consider the recommendation procedure as a static process and make recommendations following a fixed strategy. In this paper, we propose a novel recommender system with the capability of continuously improving its strategies during the interactions with users. We model the sequential interactions between users and a recommender system as a Markov Decision Process (MDP) and leverage Reinforcement Learning (RL) to automatically learn the optimal strategies via recommending trial-and-error items and re- ceiving reinforcements of these items from users' feedback. Users' feedback can be positive and negative and both types of feedback have great potentials to boost recommendations. However, the number of negative feedback is much larger than that of positive one; thus incorporating them simultaneously is challenging since positive feedback could be buried by negative one. In this paper, we develop a novel approach to incorporate them into the proposed deep recommender system (DEERS) framework. The experimental results based on real-world e-commerce data demonstrate the ef- fectiveness of the proposed framework. Further experiments have been conducted to understand the importance of both positive and negative feedback in recommendations.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

Traditional recommender systems consider the recommendation procedure as a static process and make recommendations following a fixed strategy.

Proposed recommender system with the

- capability of continuously improving its strategies during the interactions with users.
- model the sequential interactions between users and a recommender system as a Markov Decision Process (MDP) and
- leverage Reinforcement Learning (RL)
    - to automatically learn the optimal strategies via recommending trial-and-error items and
    - receiving reinforcements of these items from users' feedback.
- Incorporate highly imbalance user feedback into the proposed deep recommender system (DEERS) framework.

The experimental results based on real-world e-commerce data demonstrate the effectiveness of the proposed framework. Further experiments have been conducted to understand the importance of both positive and negative feedback in recommendations.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

Figure 1: Impact of negative feedback on recommendations.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

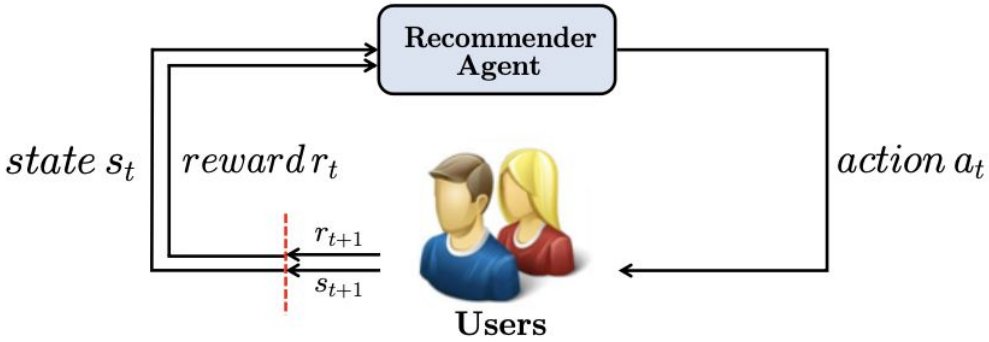


Figure 2: The agent-user interactions in MDP.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

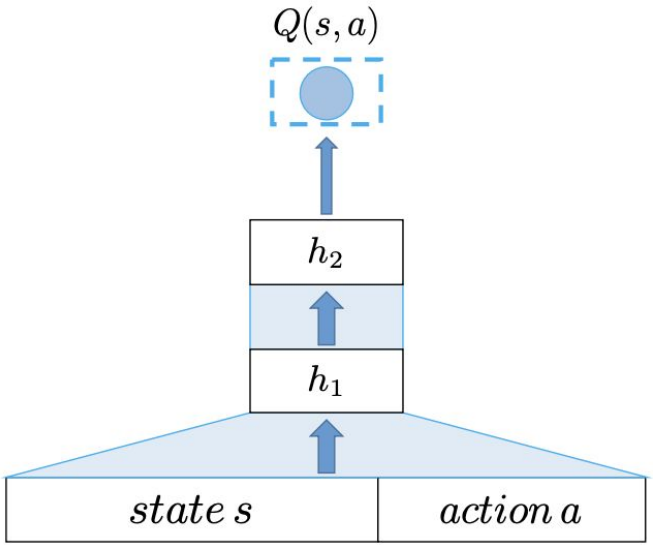


Figure 3: The architecture of the basic DQN model for recommendations.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.3 Recommendations with Negative Feedback via Pairwise Deep RL

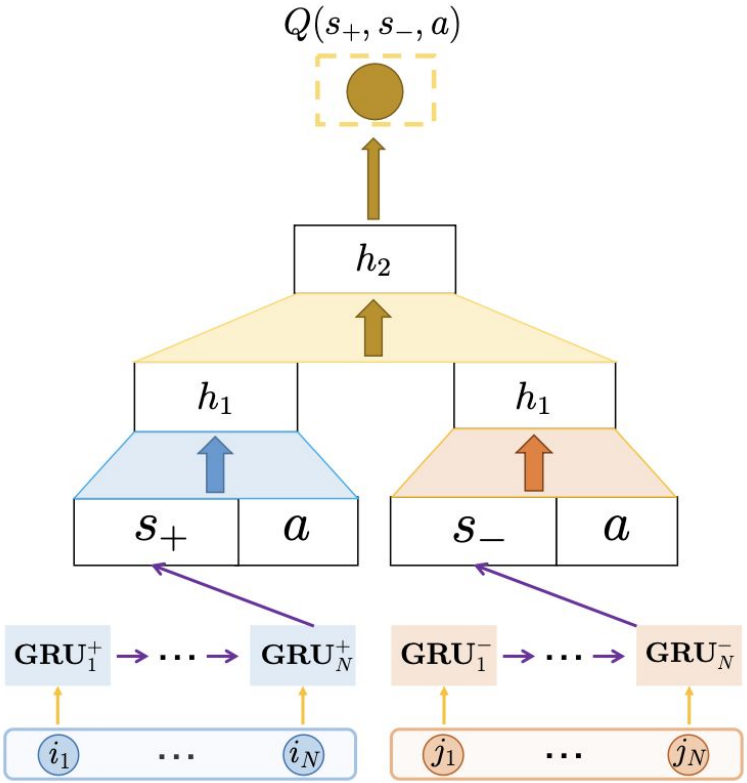


Figure 4: The architecture of the DEERS framework.

[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL

Conversational recommender systems (CRS) enable the traditional recommender systems to explicitly acquire user preferences to- wards items and attributes through interactive conversations. Reinforcement learning (RL) is widely adopted to learn conversational recommendation policies to decide what attributes to ask, which items to recommend, and when to ask or recommend, at each conversation turn. However, existing methods mainly target at solving one or two of these three decision-making problems in CRS with separated conversation and recommendation components, which restrict the scalability and generality of CRS and fall short of providing a stable training procedure. To tackle these challenges, we propose to formulate these three decision-making problems in CRS as a unified policy learning task. In order to systematically integrate conversation and recommendation components, we develop a dynamic weighted graph based RL method to learn a policy to select the action at each conversation turn, either asking an attribute or recommending items. Further, to deal with the sample efficiency issue, we propose two action selection strategies for reducing the candidate action space according to the preference and entropy information. Experimental results on two benchmark CRS datasets and a real-world E-Commerce application show that the proposed method not only significantly outperforms state-of-the-art methods but also enhances the scalability and stability of CRS.

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL

Conversational recommender systems (CRS) enable the traditional recommender systems
- to explicitly acquire user preferences towards items and attributes through interactive conversations.

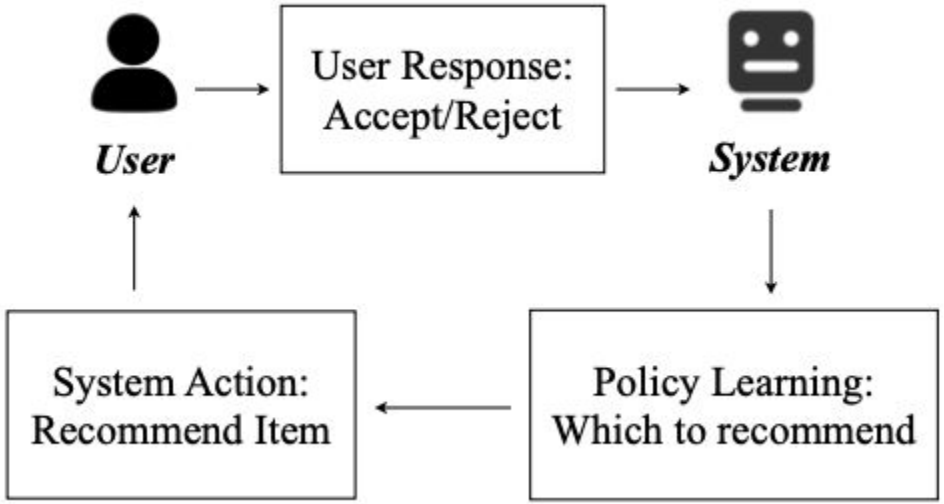
Reinforcement learning (RL) is widely adopted to learn conversational recommendation policies
- to decide what attributes to ask, which items to recommend, and
- when to ask or recommend, at each conversation turn.

Existing methods mainly
- target at solving one or two of these three decision-making problems in CRS
- with separated conversation and recommendation components,
- which restrict the scalability and generality of CRS and f
- all short of preserving a stable training procedure.

Propose to formulate these three decision-making problems in CRS as a unified policy learning task.
Systematically integrate conversation and recommendation components,
- develop a dynamic weighted graph based RL method to learn a policy to select the action at each conversation turn,
    - either asking an attribute or recommending items.
- To deal with the sample efficiency issue, proposed two action selection strategies for reducing the candidate action space according to the preference and entropy information.

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

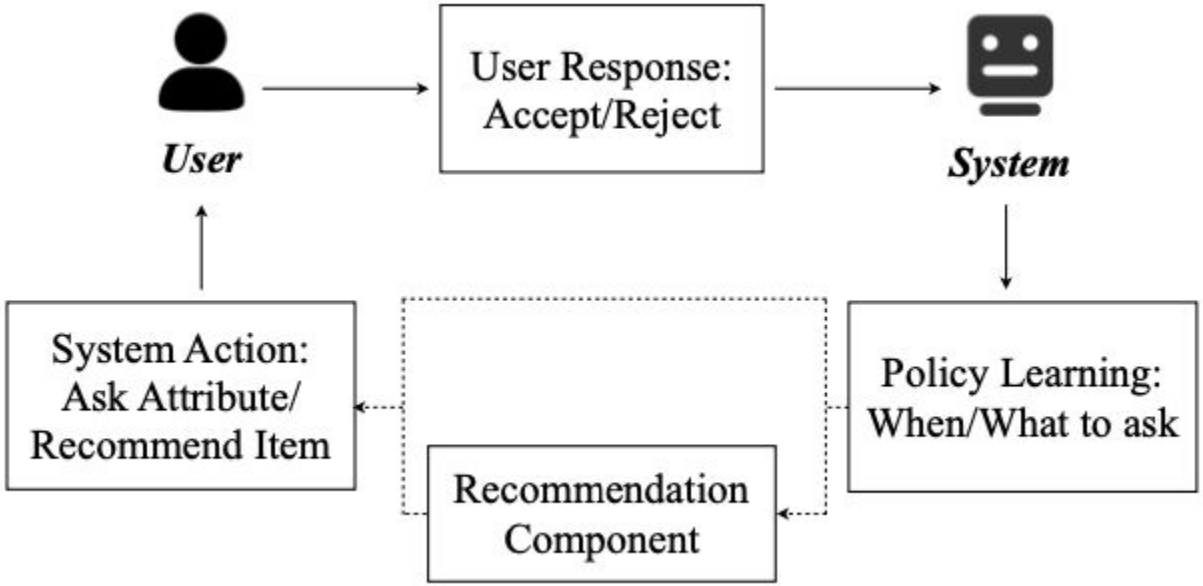
3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL



a) Interactive Recommender System

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

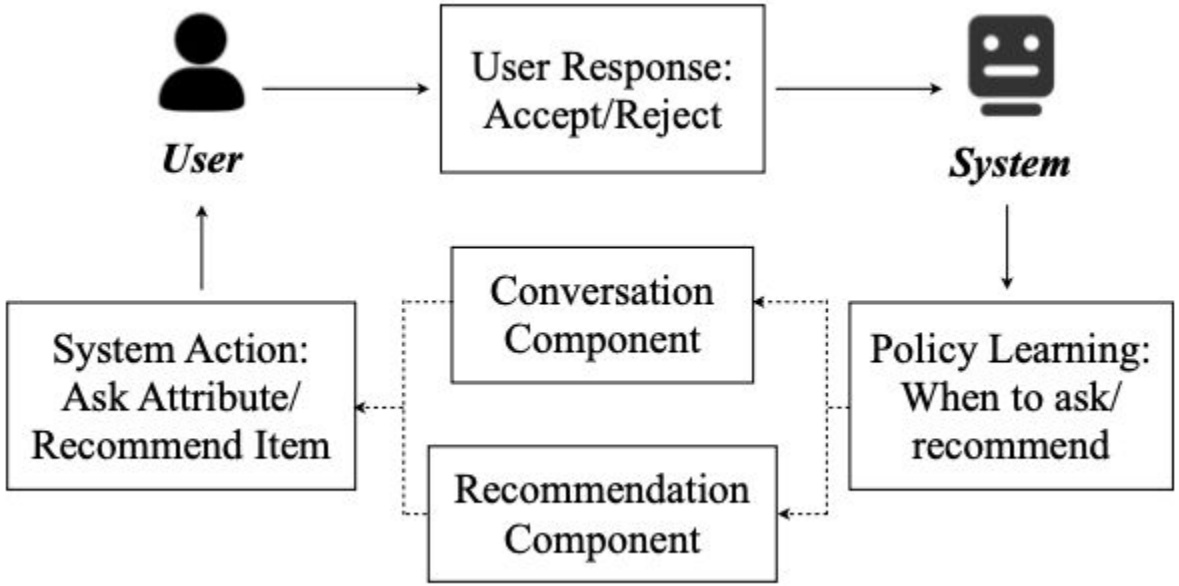
3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL



(b) Conversational Recommender System, e.g., CRM, EAR

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL



(c) Conversational Recommender System, e.g., SCPR

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL

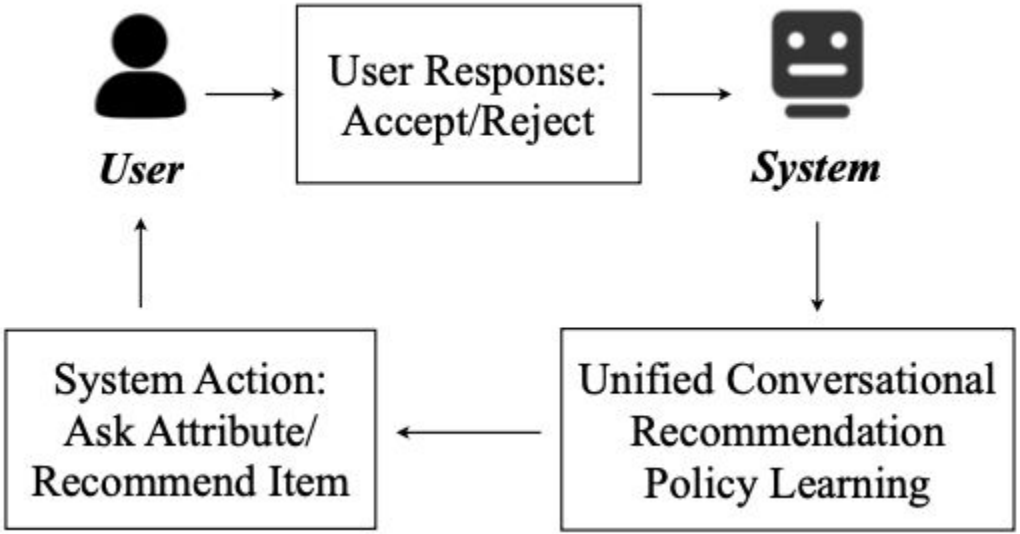


Figure 2: Illustration of unified policy learning for CRS.

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

3.4 Unified Conversational Recommendation Policy Learning via Graph-based RL

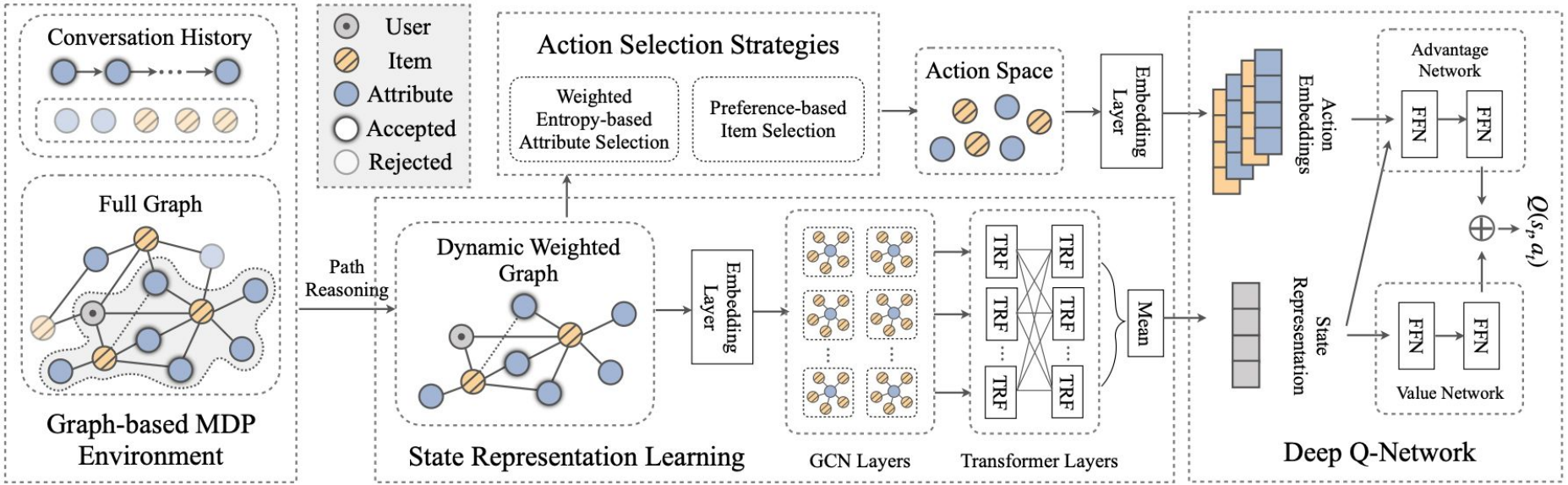


Figure 3: Overview of the proposed method, UNICORN, for unified conversational recommendation policy learning.

*__UNI__fied __CO__nversational __R__ecomme__N__der (UNICORN)

[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

4 DRL approaches for CRS - III (20min)

- Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender [11]
- Large-scale Interactive Recommendation with Tree-structured Policy Gradient [12]
- A Reinforcement Learning Framework for Interactive Recommendation [13]
- Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework [14]

4.1 Exploiting Deep Learning and Hierarchical Reinforcement Learning for CRS

In this paper, we propose a framework based on Hierarchical Reinforcement Learning for dialogue management in a Conversational Recommender System scenario. The framework splits the dialogue into more manageable tasks whose achievement corresponds to goals of the dialogue with the user. The framework consists of a meta-controller, which receives the user utterance and understands which goal should pursue, and a controller, which exploits a goal-specific representation to generate an answer composed by a sequence of tokens. The modules are trained using a two-stage strategy based on a preliminary Supervised Learning stage and a successive Reinforcement Learning stage.

11] Greco C., et. al., Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender Systems, 2017

4.1 Exploiting Deep Learning and Hierarchical Reinforcement Learning for CRS

Propose a framework based on

- Hierarchical Reinforcement Learning for dialogue management in a Conversational Recommender System scenario.

The framework

- splits the dialogue into more manageable tasks whose achievement corresponds to goals of the dialogue with the user.

The framework consists of

- a meta-controller, which receives the user utterance and understands which goal should pursue, and
- a controller, which exploits a goal-specific representation to generate an answer composed by a sequence of tokens.

The modules are trained using a two-stage strategy based on a preliminary Supervised Learning stage and a successive Reinforcement Learning stage.

11] Greco C., et. al., Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender Systems, 2017

4.1 Exploiting Deep Learning and Hierarchical Reinforcement Learning for CRS

1. A framework based on HRL in which each CRS goal is modeled as a goal-specific representation module which learns a useful representation for the given goal;
2. An answer generation module leveraging the learned goal-specific representations and the user preferences to generate appropriate answers.

11] Greco C., et. al., Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender Systems, 2017

4.1 Exploiting Deep Learning and Hierarchical Reinforcement Learning for CRS

11] Greco C., et. al., Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender Systems, 2017

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

Reinforcement learning (RL) has recently been introduced to interactive recommender systems (IRS) because of its nature of learning from dynamic interactions and planning for long- run performance. As RL is always with thousands of items to recommend (i.e., thousands of actions), most existing RL- based methods, however, fail to handle such a large discrete action space problem and thus become inefficient. The existing work that tries to deal with the large discrete action space problem by adopting the ranking score/item embedding framework suffers from the inconsistency between the continuous action representation (the output of the actor net- work) and the real discrete action. To avoid such inconsistency and achieve high efficiency and recommendation effectiveness, in this paper, we propose a Tree-structured Policy Gradient Recommendation (TPGR) framework, where a balanced hierarchical clustering tree is built over the items and picking an item is formulated as seeking a path from the root to a certain leaf of the tree. Extensive experiments on carefully-designed environments based on two real-world datasets demonstrate that our model provides superior recommendation performance and significant efficiency improvement over state-of-the-art methods.

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

Reinforcement learning (RL) has recently been introduced to interactive recommender systems (IRS) because of

- its nature of learning from dynamic interactions and
- planning for long-run performance.

As IRS is always with thousands of items to recommend (i.e., thousands of actions), most existing RL- based methods, however, fail to handle such a large discrete action space problem and thus become inefficient.

The existing work that tries to deal with the large discrete action space problem by utilizing the deep deterministic policy gradient framework suffers from the inconsistency between the continuous action representation (the output of the actor net- work) and the real discrete action.

To avoid such inconsistency and achieve high efficiency and recommendation effectiveness, in this paper, we propose a Tree-structured Policy Gradient Recommendation (TPGR) framework, where

- a balanced hierarchical clustering tree is built over the items and picking an item is formulated as seeking a path from the root to a certain leaf of the tree.

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

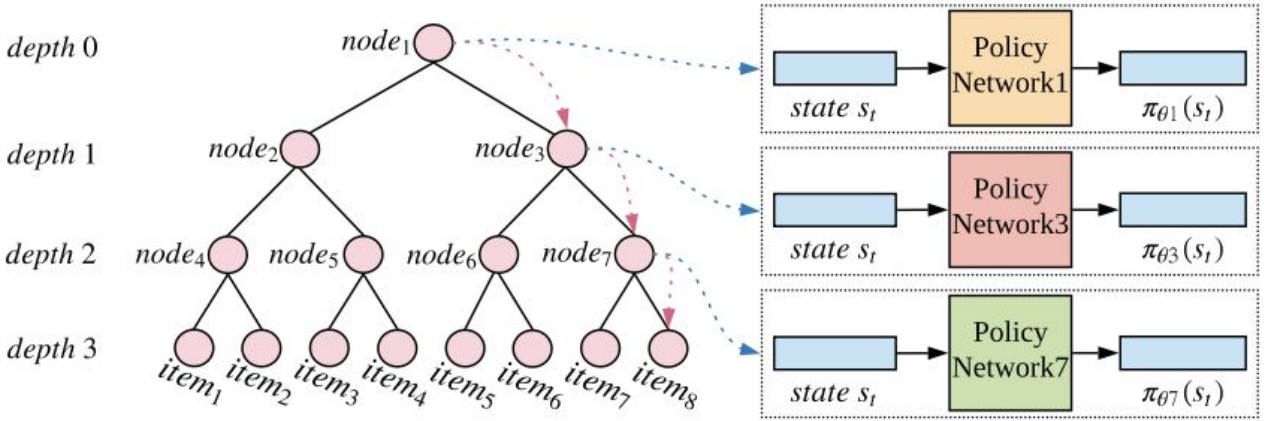


Figure 1: Architecture of TPGR.

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

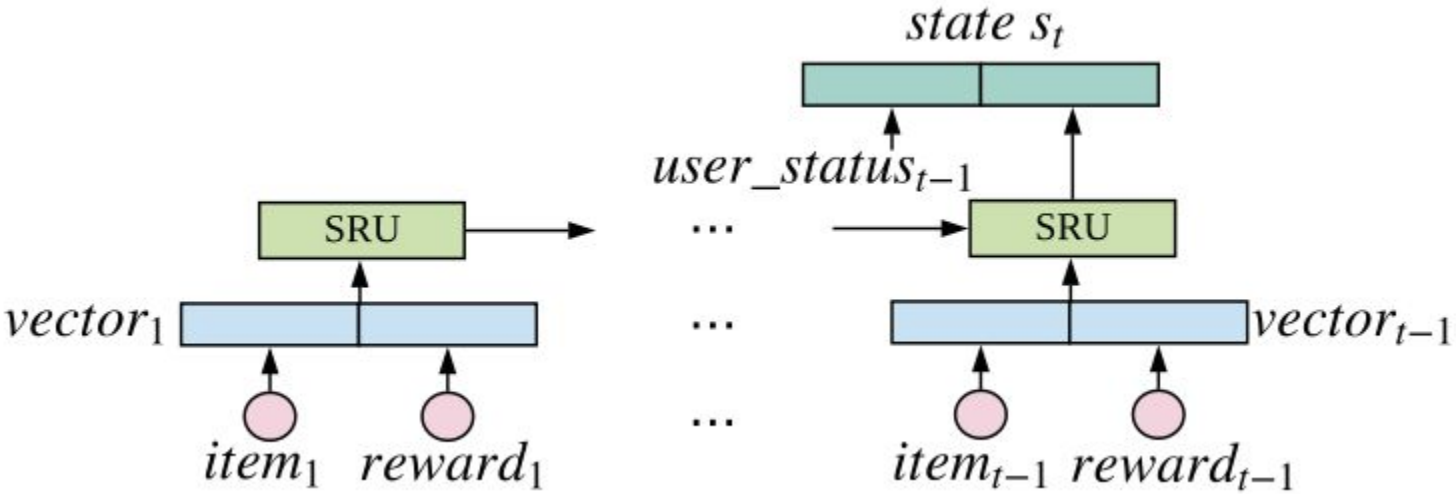


Figure 2: State representation.

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.2 Large-scale Interactive Recommendation with Tree-structured Policy Gradient

Extensive experiments on carefully-designed environments based on two real-world datasets demonstrate that our model provides superior recommendation performance and significant efficiency improvement over state-of-the-art methods.

[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019

4.3 Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation

Applying reinforcement learning (RL) in recommender systems is attractive but costly due to the constraint of the interaction with real customers. Hence, performing online policy learning through interacting with real customers usually harms customer experiences. A practical alternative is to build a recommender agent offline from logged data, whereas directly using logged data offline leads to the problem of selection bias between logging policy and the recommendation policy. The existing direct offline learning algorithms, such as Monte Carlo methods and temporal difference methods are either computationally expensive or unstable on convergence.

To address these issues, we propose Pseudo Dyna-Q (PDQ). In PDQ, instead of interacting with real customers, we resort to a customer simulator, referred to as the World Model, which is designed to simulate the environment and handle the selection bias of logged data. During policy improvement, the World Model is constantly updated and optimized adaptively, according to the current recommendation policy. This way, the proposed PDQ not only avoids the instability of convergence and high computation cost of existing approaches but also provides unlimited interactions without involving real customers. Moreover, a proved upper bound of empirical error of reward function guarantees that the learned offline policy has lower bias and variance. Extensive experiments demonstrated the advantages of PDQ on two real-world datasets against state-of-the-arts methods.

[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020

4.3 Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation

Applying reinforcement learning (RL) in recommender systems is attractive but

- costly due to the constraint of the interaction with real customers,
- where performing online policy learning through interacting with real customers usually harms customer experiences.

A practical alternative is to

- build a recommender agent offline from logged data,
- whereas directly using logged data offline leads to the problem of selection bias between logging policy and the recommendation policy.

The existing direct offline learning algorithms, such as Monte Carlo methods and temporal difference methods are either computationally expensive or unstable on convergence.

Propose Pseudo Dyna-Q (PDQ). In PDQ, instead of interacting with real customers,

- resort to a customer simulator, referred to as the World Model, which is designed to simulate the environment and handle the selection bias of logged data.
- During policy improvement, the World Model is constantly updated and optimized adaptively, according to the current recommendation policy.

This way, the proposed PDQ not only avoids the instability of convergence and high computation cost of existing approaches but also provides unlimited interactions without involving real customers.
Moreover, a proved upper bound of empirical error of reward function guarantees that the learned offline policy has lower bias and variance.

[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020

4.3 Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation

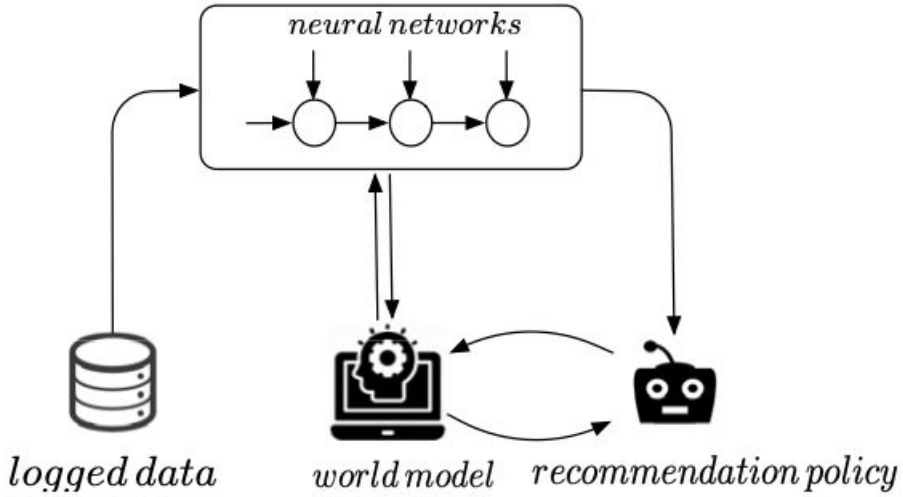
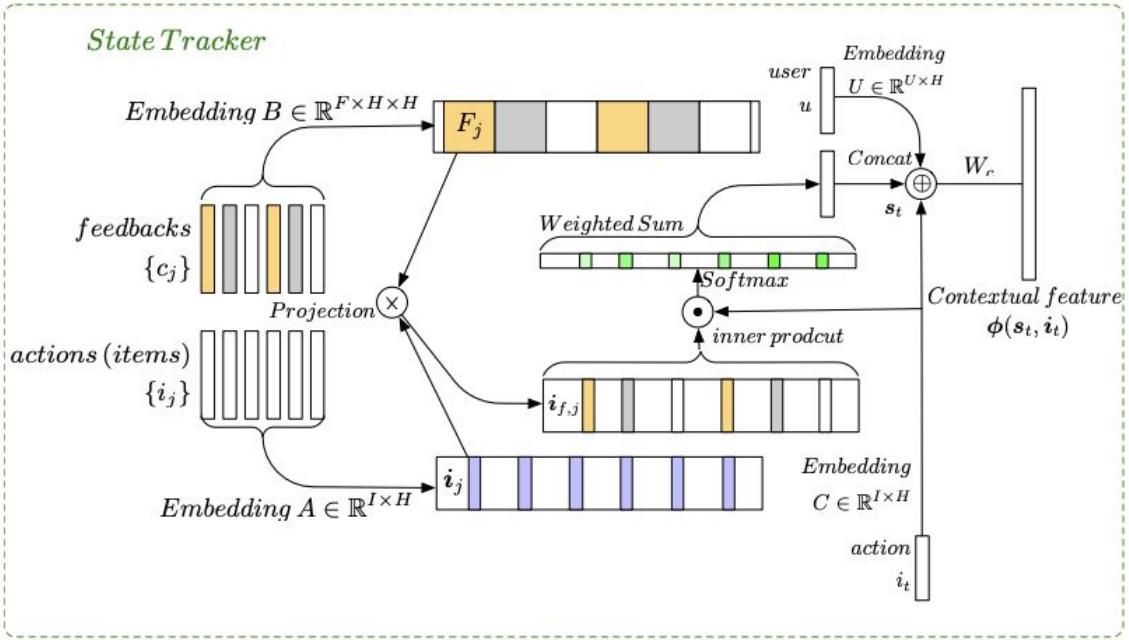


Figure 1: An illustration of Pseudo Dyna-Q Framework.

[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020

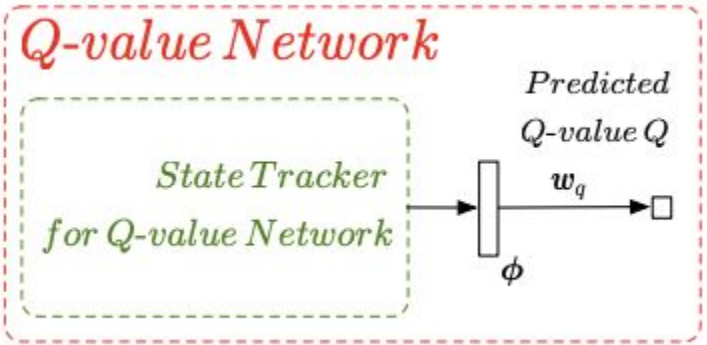
4.3 Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation



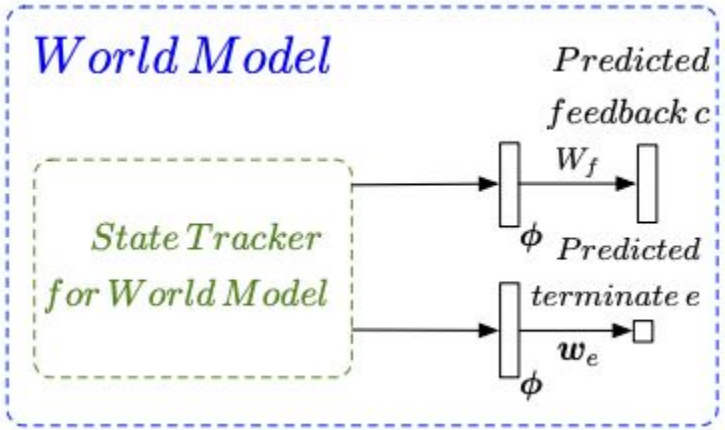
The neural architecture of PDQ. (a)The state tracker maintains customer's preferences with a memory network and self-attention mechanism

[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020

4.3 Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation



The neural architecture of PDQ. (b)The Q-value Network predicts the Q-value by the inner product between tracked customer's preferences and a weight vector



(c)The world model generates customer's feedback with a multi-head MLP.

[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020

4.4 Large-scale Interactive CRS using Actor-Critic Framework

We propose AC-CRS, a novel conversational recommendation system based on reinforcement learning that better models user interaction compared to prior work. Interactive recommender systems expect an initial request from a user and then iterate by asking questions or recommending potential matching items, continuing until some stopping criterion is achieved. Unlike most existing works that stop as soon as an item is recommended, we model the more realistic expectation that the interaction will continue if the item is not appropriate. Using this process, AC-CRS is able to support a more flexible conversation with users. Unlike existing models, AC- CRS is able to estimate a value for each question in the conversation to make sure that questions asked by the agent are relevant to the target item (i.e., user needs). We also model the possibility that the system could suggest more than one item in a given turn, allowing it to take advantage of screen space if it is present. AC-CRS also better accommodates the massive space of items that a real-world recommender system must handle. Experiments on real-world user purchasing data show the effectiveness of our model in terms of standard evaluation measures such as NDCG.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

4.4 Large-scale Interactive CRS using Actor-Critic Framework

Propose AC-CRS, a novel conversational recommendation system based on reinforcement learning that better models user interaction.
Interactive recommender systems

- expect an initial request from a user and
- then iterate by asking questions or recommending potential matching items,
- continuing until some stopping criterion is achieved.

Unlike most existing works that stop as soon as an item is recommended,

- we model the more realistic expectation that the interaction will continue if the item is not appropriate.

Using this process, AC-CRS is able to

- support a more flexible conversation with users.
- estimate a value for each question in the conversation to make sure that questions asked by the agent are relevant to the target item (i.e., user needs).

We also model the possibility that the system could suggest more than one item in a given turn, allowing it to take advantage of screen space if it is present.
AC-CRS also better accommodates the massive space of items that a real-world recommender system must handle.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

4.4 Large-scale Interactive CRS using Actor-Critic Framework

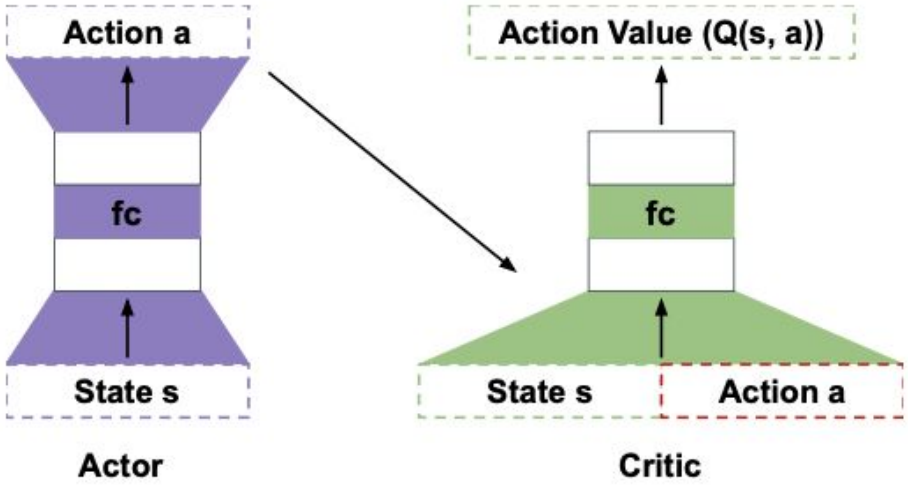


Figure 1: The Actor-Critic framework.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

4.4 Large-scale Interactive CRS using Actor-Critic Framework

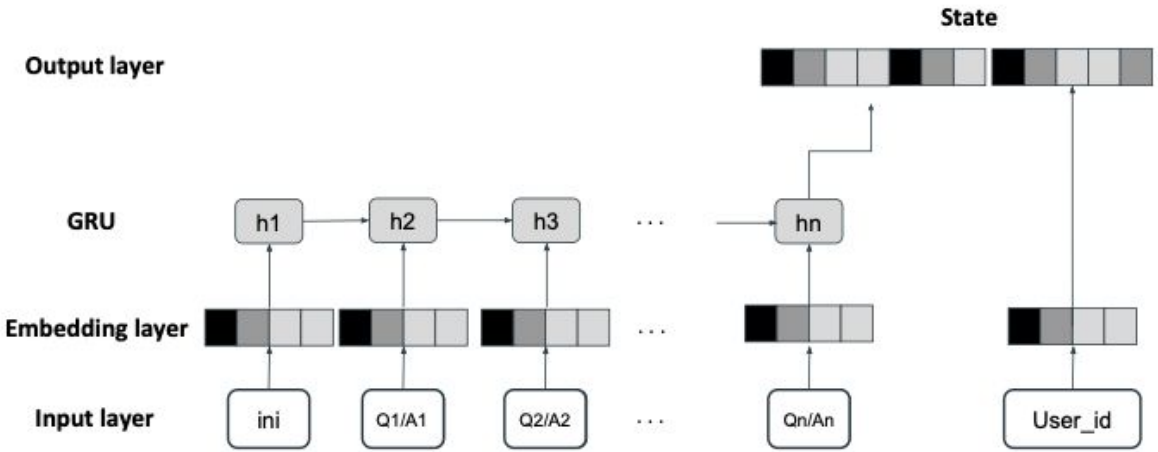


Figure 2: The model for generating the State.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

4.4 Large-scale Interactive CRS using Actor-Critic Framework

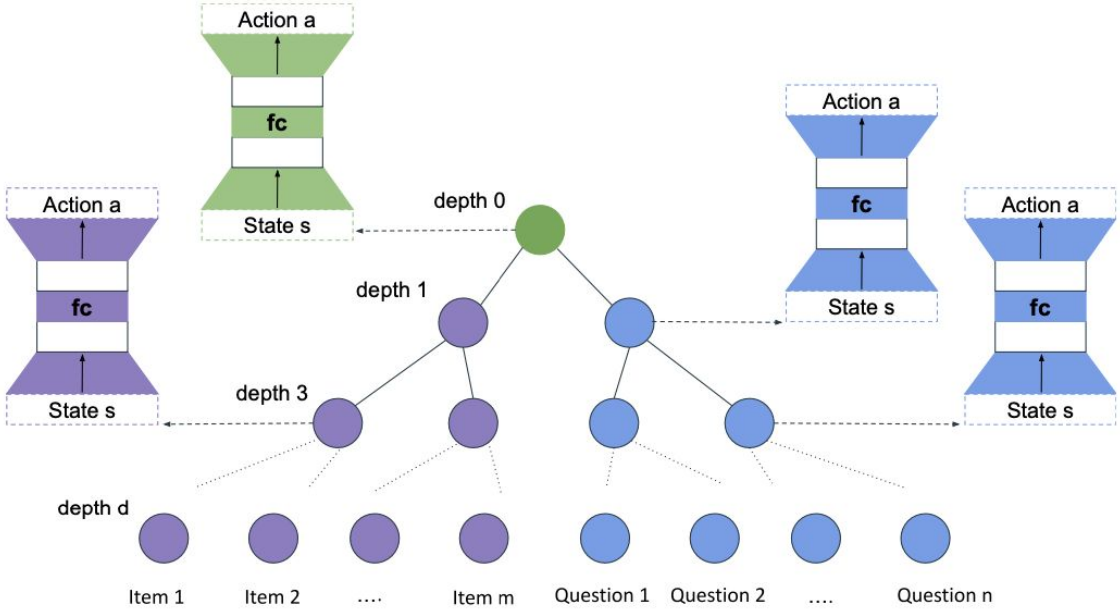


Figure 3: The Architecture of tree structured Actor.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

4.4 Large-scale Interactive CRS using Actor-Critic Framework

Experiments on real-world user purchasing data show the effectiveness of our model in terms of standard evaluation measures such as NDCG.

[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

REFERENCES

[1] Richard Sutton et. al., Reinforcement Learning an Introduction 2nd Edition (Book)
[2] David Silver, Introduction to Reinforcement Learning (Course)
[3] Sun Y. and Zhang Y., Conversational Recommender System, SIGIR 2018
[4] Li R., et. al., Towards Deep Conversational Recommendations, NIPS 2018
[5] Lei W., et. al., Estimation–Action–Reflection: Towards Deep Interaction Between Conversational and Recommender Systems, WSDM 2020
[6] Learning and Adaptivity in Interactive Recommender Systems, ICEC 2007

[7] Zhou K., et. al., Towards Topic-Guided Conversational Recommender System
[8] Lei W., et. al, Interactive Path Reasoning on Graph for Conversational Recommendation, KDD 2020
[9] Zhao X., et. al., Recommendations with Negative Feedback via Pairwise Deep Reinforcement Learning, KDD 2018
[10] Deng Y., et. al., Unified Conversational Recommendation Policy Learning via Graph-based Reinforcement Learning, SIGIR 2021

[11] Greco C., et. al., Exploiting Deep Learning and Hierarchical Reinforcement Learning for Conversational Recommender Systems, 2017
[12] Chen H., et. al., Large-scale Interactive Recommendation with Tree-structured Policy Gradient, AAAI 2019
[13] Zou L., et. al., Pseudo Dyna-Q: A Reinforcement Learning Framework for Interactive Recommendation, WSDM 2020
[14] Montazeralghaem A., et. al. Large-scale Interactive Conversational Recommendation System using Actor-Critic Framework, RecSys 2021

Author’s Hands on Tutorial
[15] Sonie Omprakash., et. al, concept2code: Deep Reinforcement Learning for Conversational AI, KDD 2022

Thank You.

omisonie@gmail.com

www.DeepThinking.AI

Overview

TUTORIAL
- 90 min

ABSTRACT

Deep Reinforcement Learning (DRL) uses best of both Reinforcement Learning and Deep Learning for solving problems which can not be addressed by them individually. Deep Reinforcement Learning has been used widely for games, robotics etc. Limited work has been done for applying DRL for Conversational Recommender System (CRS). Hence, in this tutorial cover application of DRL for CRS.
We give conceptual introduction to Reinforcement Learning and Deep Reinforcement Learning and cover Deep Q-Network, Dyna, REINFORCE and Actor Critic methods.
We then cover various real life case studies with increasing complexity starting from CRS, deep CRS, adaptivity, topic guided CRS, deep and large scale CRSs.
We plan to share pre-read for Reinforcement Learning and Deep Reinforcement learning so that participants are able to grasp the material well.

TARGET AUDIENCE
- introductory and intermediate

PREREQUISITE KNOWLEDGE OR SKILLS

Have college level math background and desire to understand
high level math and intuition behind the techniques.
Have applied understanding of machine learning, deep learning
and reinforcement learning terminologies and have gone through
few projects in Python using Jupyter notebook.
Participant can refer to our prior tutorial at KDD-2022 [15].